

人工智能原理与应用

詹紫琳

邮箱: zilin.zhan@szu.edu.cn

Fall 2025



深圳大学
SHENZHEN UNIVERSITY

回顾

单智能体 (single-agent) 有搜索

- 目标确定：智能体只需找到代价最小或最优解
- 单一玩家，环境中没有对手：不需要考虑别人会怎么回应

任务环境	可观测	智能体	确定性	回合式	静态	离散
填字游戏	完全	单	确定性	序贯	静态	离散
限时国际象棋	完全	多	确定性	序贯	半动态	离散
扑克	部分	多	非确定性	序贯	静态	离散
西洋双陆棋	完全	多	非确定性	序贯	静态	离散
驾驶出租车	部分	多	非确定性	序贯	动态	连续
医疗诊断	部分	单	非确定性	序贯	动态	连续
图片分析	完全	单	确定性	回合式	半动态	连续
零件选取机器人	部分	单	非确定性	回合式	动态	连续
提炼厂控制器	部分	单	非确定性	序贯	动态	连续
交互英语教师	部分	多	非确定性	序贯	动态	离散

多智能体决策

- 多智能体环境：存在多个具有行为能力与目标的决策主体（智能体）。
- 竞争环境：智能体之间的目标存在对立性，一个智能体的收益往往意味着另一个的损失。
- 竞争改变了搜索的目标函数：不再是最大化我自身收益，而是要**同时**预期对手如何最优反制我。

博弈搜索

- 博弈 (Game) : 多个决策者在共享环境中通过行动影响收益的过程。
- 博弈论 (Game Theory) : 用于研究策略选择与均衡结果的数学框架
- 零和 (Zero-Sum) 博弈: 一个玩家的收益 = 另一个玩家的损失
 - 象棋、国际象棋、扑克、乒乓球、篮球
- 偶发性 (Stochasticity) : 确定性 vs 随机游戏

类型	定义	示例	搜索模型
确定性游戏	行动后结果唯一确定	国际象棋、围棋	Minimax
随机游戏 (含偶发性)	行动结果存在概率分布	扑克 (不完全信息)、掷骰子游戏	Expectiminimax

博弈搜索



• 零和博弈

- 智能体有相反的输出（结果数值）
- 己方最大化，敌方最小化
- 对抗性的，纯粹的竞争，无合作

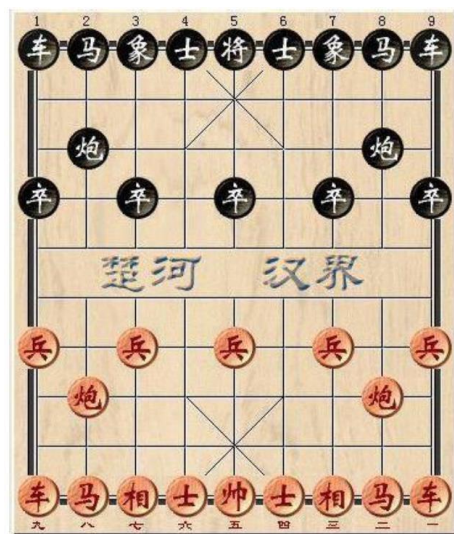
• 一般游戏

- 智能体有不同的输出（结果数值）
- 合作、竞争或袖手旁观都有可能

博弈搜索

- 确定性游戏的问题定义：
 - 状态States: S (开始于 s_0)
 - 玩家Players: $P=\{1...N\}$ (通常轮流)
 - 动作Actions: A (可能取决于玩家/状态)
 - 状态转移: $S \times A \rightarrow S$
 - 终止测试: $S \rightarrow \{t, f\}$ (判断博弈是否结束的规则。；当测试结果为真(t)时，游戏停止)
 - 终止效用: $S \times P \rightarrow R$ (在博弈终止时，为每个玩家分配的数值奖励(或惩罚)，这是玩家的目标函数)
- 玩家的解决方案是一项策略Strategy: $S \rightarrow A$ (对于任何可能遇到的状态 S ，我应该执行哪个动作 A)
 - 在单Agent搜索中，解决方案是一系列动作的序列(比如：上-上-右-下)。

博弈搜索



巨大的状态空间

九列十行

十四种不同的棋子

(黑、红不同类) 三十二个棋子

普通搜索（如路径规划）	博弈搜索（如棋类）
深度通常较小（十几到几十）	深度往往非常大（几十到上百手）
终止条件明确（比如到达目标）	终止难以提前判断，且双方会拖延与反制
AI 只需计划自己的路径	AI 必须“预测 + 反预测 + 再反预测”

博弈搜索

如何应对巨大的状态空间？

通常，博弈搜索树的分支因子和终止状态的深度较大。

- 状态数

中国象棋大约在 10^{40} 量级。

- 博弈搜索树：

- 搜索树的深度影响性能。

- 平均分支因子的层数次幂作为博弈树大小的一个估算上限值。

- 国际象棋搜索树分支因子约35，路径深度约100。如果考虑完整的搜索策略，至少 35^{100} 量级，就是用超级计算机来处理，所需时间也是天文数字。

- 博弈要求在无法计算出最优决策的情况下也要给出某种决策，所以必尽可能地利用好时间。

博弈搜索

如何应对巨大的状态空间？

- 很多游戏程序基于
 - 剪枝 + 迭代加深 + 庞大的数据库 + ...
- 尽管游戏程序的方法很普通，但是通过许多专门调整的增强功能（例如，**评价函数**）大大改善了它们的实现。
- 比如Go (围棋)的分支因子过高。游戏程序必须依赖**庞大的数据库和模式识别技术**。
- **搜索**非常重要！

博弈搜索

剪枝 (Pruning) —— 减少不必要的搜索

典型技术: Alpha-Beta 剪枝

在 Minimax 搜索中, 当已知某条分支不会影响最终决策时, 直接停止展开
不改变决策结果, 只减少计算量

目标: 减少节点扩展/ 降低策略质量

博弈搜索

迭代加深 (Iterative Deepening) —— 在时间限制下尽量深入过程:

- 先搜索深度 1
- 再搜索深度 2
- 再搜索深度 3
- ...

每次搜索都保留当前**最好动作**作为候选解。

- 即使时间被切断（棋钟时间到），仍可**立刻返回最优已知策略**
- 非常适用于**实时决策**（象棋引擎、实时对抗游戏、自动驾驶）

博弈搜索

评估函数 (Evaluation Function) —— 在未到终局也能判断局面好坏

因为没有人真的能等 AI 把棋搜到底。

所以 AI 必须拥有：**局部局势判断能力**

举例：国际象棋的评估函数会考虑：

- 子力价值（王、后、车、象...）
- 控制格子数量
- 王是否暴露
- 是否形成攻击/防守结构

评估函数越好 → 不必搜索得很深也能下得很好。

博弈搜索

数据库 (Opening Books & Endgame Tables) —— “能查就不算”

开局库 (Opening Book)

- 开局阶段不是计算，而是查表
- 来源：人类专业棋谱 + 大规模自我对弈

残局数据库 (Endgame Tablebase)

- 当棋盘上只剩 5~7 子时，所有状态已被穷举证明
 - 这时引擎可以100% 精准地完胜或逼和
- 能查就不算，减少计算深度需求。

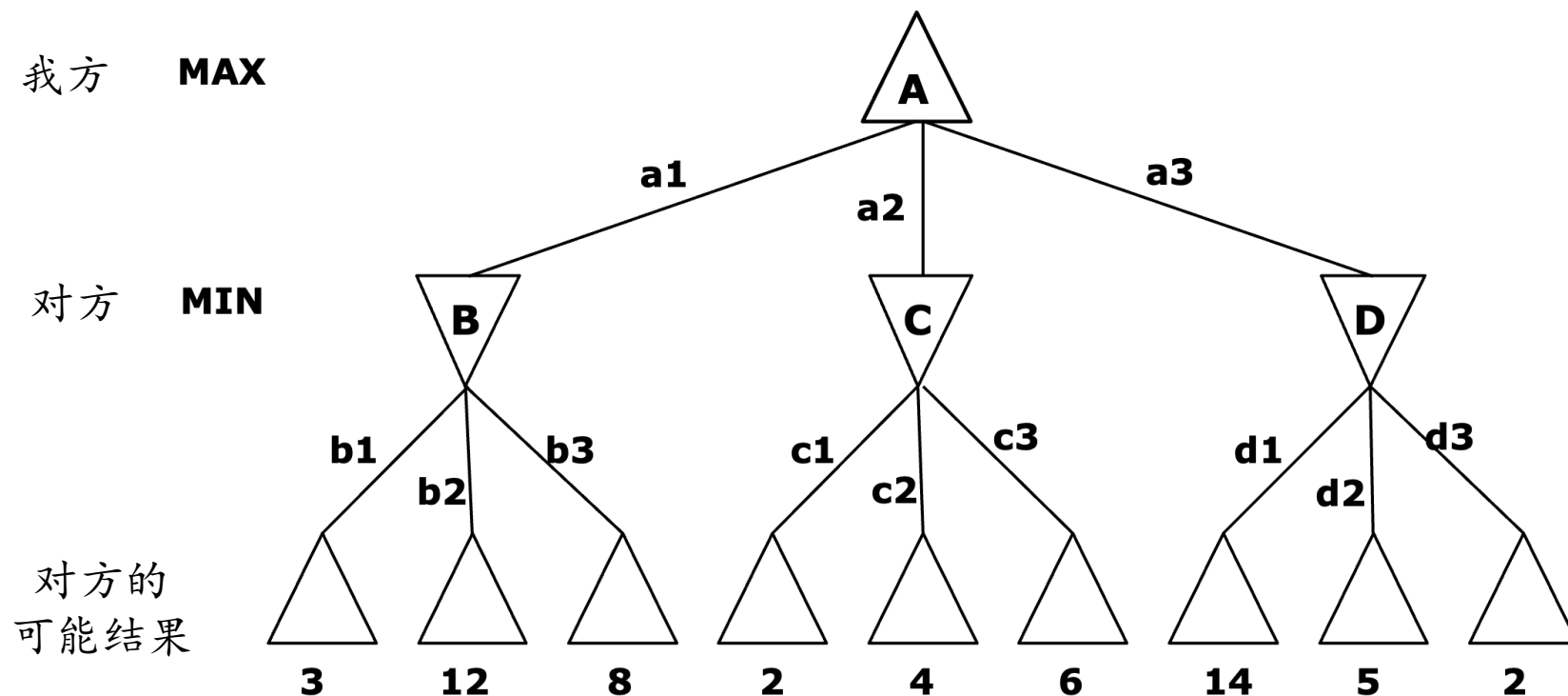
博弈搜索——MINIMAX算法

- 考虑两人博弈：MAX和MIN（为什么这样命名？）
- MAX先行，两人轮流走招，直到游戏结束。给胜者加分，给败者扣分。游戏可以形式化成含有下列组成部分的一类搜索问题：
 - S_0 ：初始状态，规范游戏开始时的情况
 - $PLAYER(s)$ ：定义s状态下该谁行动
 - $ACTIONS(s)$ ：此状态下的合法移动集合
 - $RESULT(s,a)$ ：转移模型，定义行动的结果。
 - $TERMINAL-TEST(s)$ ：终止态测试函数，游戏结束返回真，否则返回假。**游戏结束的状态称为终止状态。**
 - $UTILITY(s,p)$ ：效用函数，定义游戏者p在状态s下的数值。零和博弈是指所有棋手的收益之和在每个棋局实例中都相同。国际象棋中是0+1，1+0或 $\frac{1}{2} + \frac{1}{2}$ 。

博弈搜索——MINIMAX算法

- 博弈树的结点是状态, 边是移动
 - 结点种类包括: 初始状态、终止状态、中间状态
 - 为进行搜索需要定义函数: ACTIONS函数、UTILITY函数、RESULT函数、TERMINAL-TEST函数、PLAYER函数

博弈搜索——MINIMAX算法



MAX选择哪一步? a1/a2/a3

博弈搜索——MINIMAX算法

我方 (MAX 玩家) : 希望获得尽可能大的分值。

对手 (MIN 玩家) : 希望让我们得到尽可能小的分值 (即削弱我们) 。

因此:

MAX 层取最大值, MIN 层取最小值.

MAX 不是“单纯选最大”, 而是选 “对手最优反制后, 我们仍然能做到的最大收益

叶子节点 (最底层的数字) 是局面评估值 / 胜负好坏程度:

- 数字大 $\rightarrow$ MAX 方更有利
- 数字小 $\rightarrow$ MIN 方更有利

这些数字可以来自:

- 完全搜索到终局
- 或评估函数近似评分

博弈搜索——MINIMAX算法

技能三子棋！

博弈搜索——MINIMAX算法



MAX (X)



MIN (O)



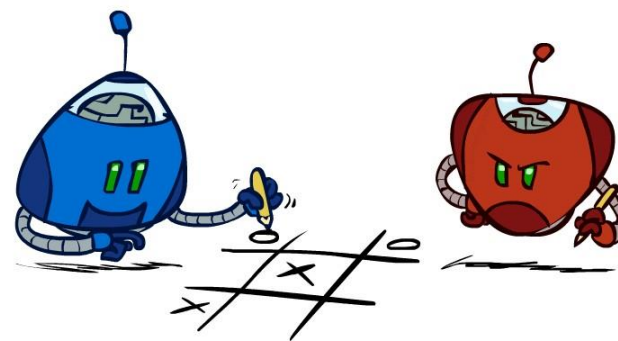
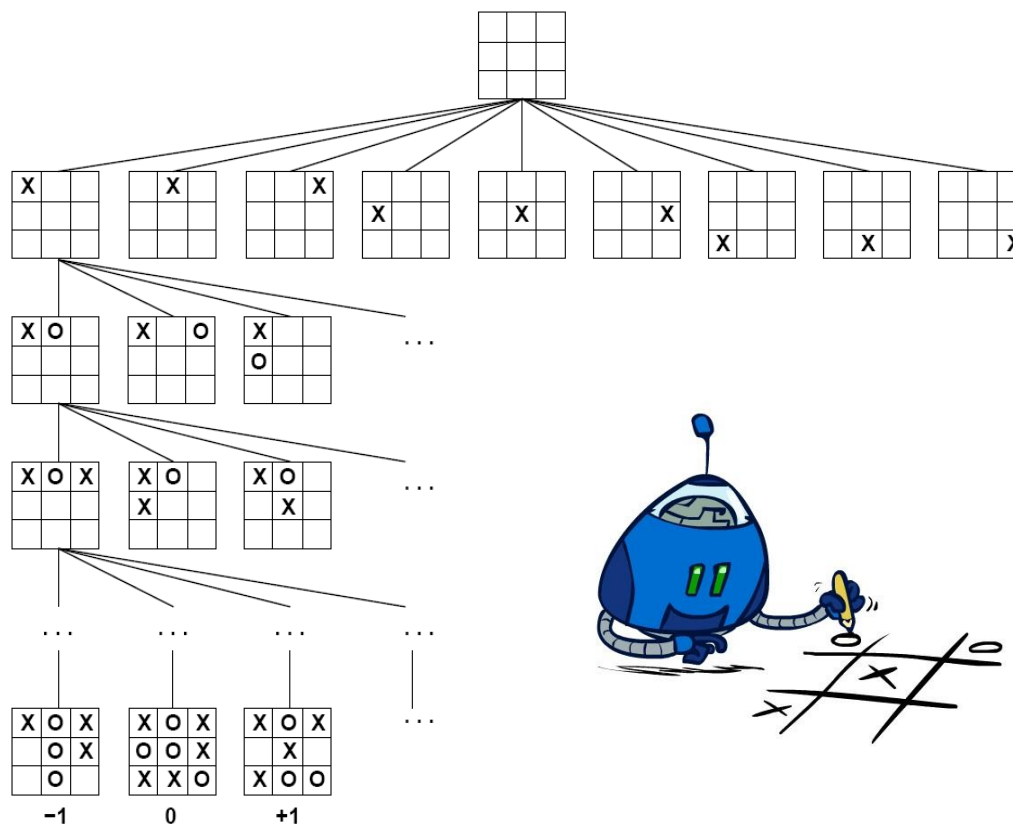
MAX (X)



MIN (O)

TERMINAL

Utility



博弈搜索——MINIMAX算法

极小极大搜索的思想

- 极小极大搜索策略是双方都从可能的走步中选一步相对好的走法执行，即在有限的搜索深度范围内进行求解。
- 怎么评价走法好不好呢？
 - 定义一个minimax值，对max有利的状态，minimax值较大，
 - 反之，对min有利的状态，minimax较小。

博弈搜索——MINIMAX算法

极小极大搜索过程

一般地：MAX代表电脑程序方

MIN代表对手方

定义一个与输赢有关的终止态效用函数 $utility(s)$ ，其函数值的大小，最终输赢来决定，比如：

MAX赢， $utility(s)$ 取+1

MAX输， $utility(s)$ 取-1

平局， $utility(s)$ 取0

博弈搜索——MINIMAX算法

极小极大搜索过程

使用效用函数和minimax函数进行估计，

必须以下述两个条件为前提：

- (1) 双方都知道各自走到什么程度、下一步可能做什么（可观测）
- (2) 不考虑偶然影响（确定性）

在这个前提下，博弈双方必须考虑：

- (1) 最优性：如何产生一个最好的走步。
- (2) 高效性：如何尽快搜索到最好的走步。

博弈搜索——MINIMAX算法

极小极大搜索过程

MINIMAX的基本思想是：

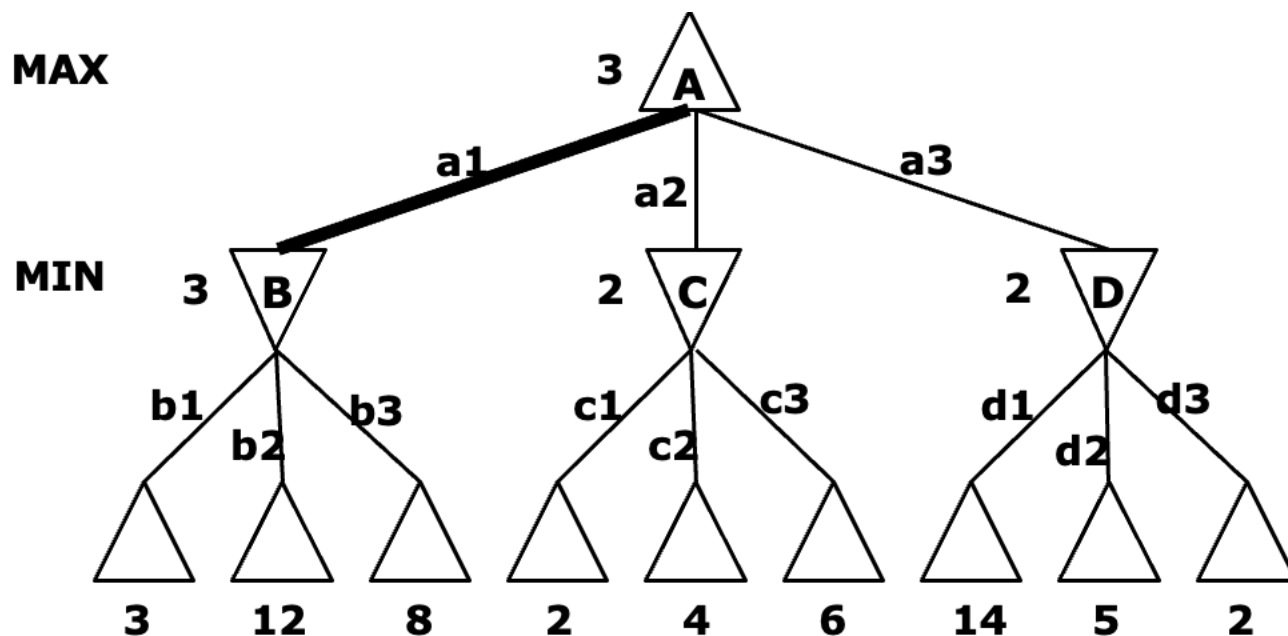
终止态的 $\text{minimax}(s)$ 函数值= 效用函数值 $\text{utility}(s)$

(1) 当轮到MIN走步的结点时，MAX应考虑最坏的情况（因此， minimax 取极小值）。

(2) 当轮到MAX走步的结点时，MAX应考虑最好的情况（因此， minimax 取极大值）。

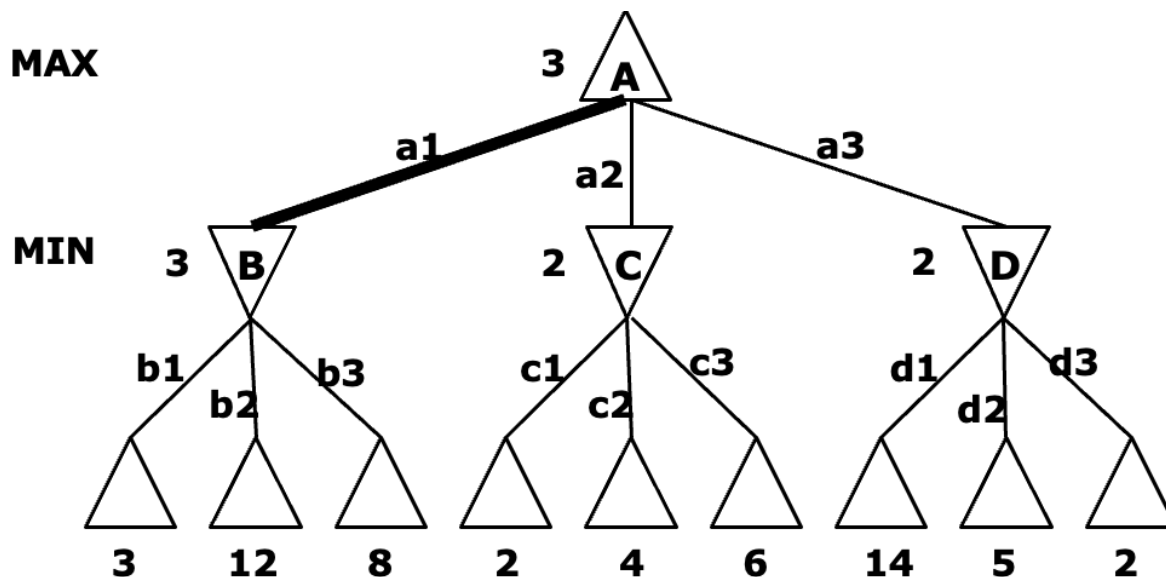
(3) 当评价往回倒推时，相应于两位棋手的对抗策略，不同层上交替的使用（1）、（2）两种方法向上传递倒推值。

博弈搜索——MINIMAX算法



△ : max 结点, ▽ : min 结点; 叶结点显示了MAX的效用函数值

博弈搜索——MINIMAX算法



MINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

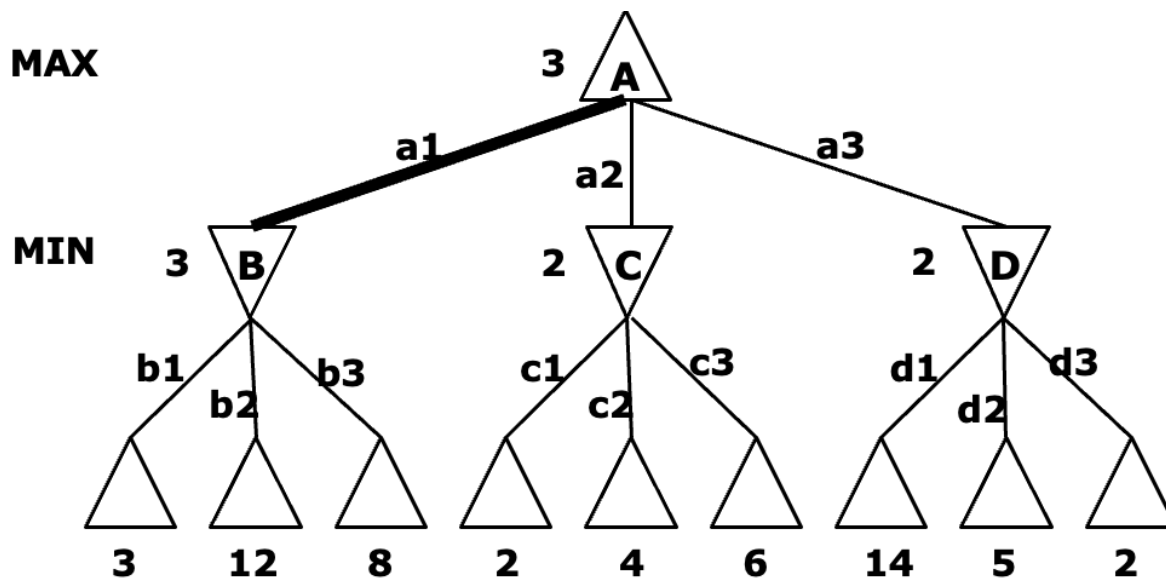
博弈搜索——MINIMAX算法

```
function MINIMAX-DECISION(state) returns an action  
  return  $\arg \max_{a \in \text{ACTIONS}(s)} \text{MIN-VALUE}(\text{RESULT}(state, a))$ 
```

```
function MAX-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow -\infty$   
  for each a in ACTIONS(state) do  
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a)))$   
  return v
```

```
function MIN-VALUE(state) returns a utility value  
  if TERMINAL-TEST(state) then return UTILITY(state)  
   $v \leftarrow \infty$   
  for each a in ACTIONS(state) do  
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a)))$   
  return v
```

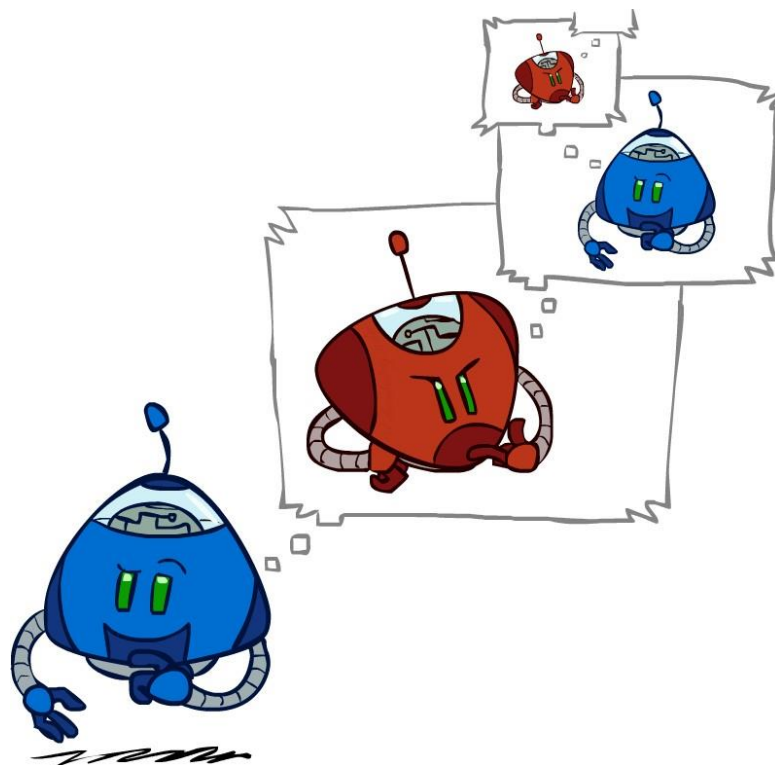
博弈搜索——MINIMAX算法



哪些结点不必搜索？
即max()的值与哪些值无关？
—— Alpha-Bate 剪枝

博弈搜索——MINIMAX算法

- MINIMAX有多有效？
 - 和DFS差不多
 - 时间复杂度: $O(b^m)$
 - 空间复杂度: $O(b^m)$
- 例:对于国际象棋, $b \approx 35$, $m \approx 100$
 - 确切的解决方案是完全不可行的
 - 但是, 我们需要探索整棵树吗?



博弈搜索——MINIMAX算法

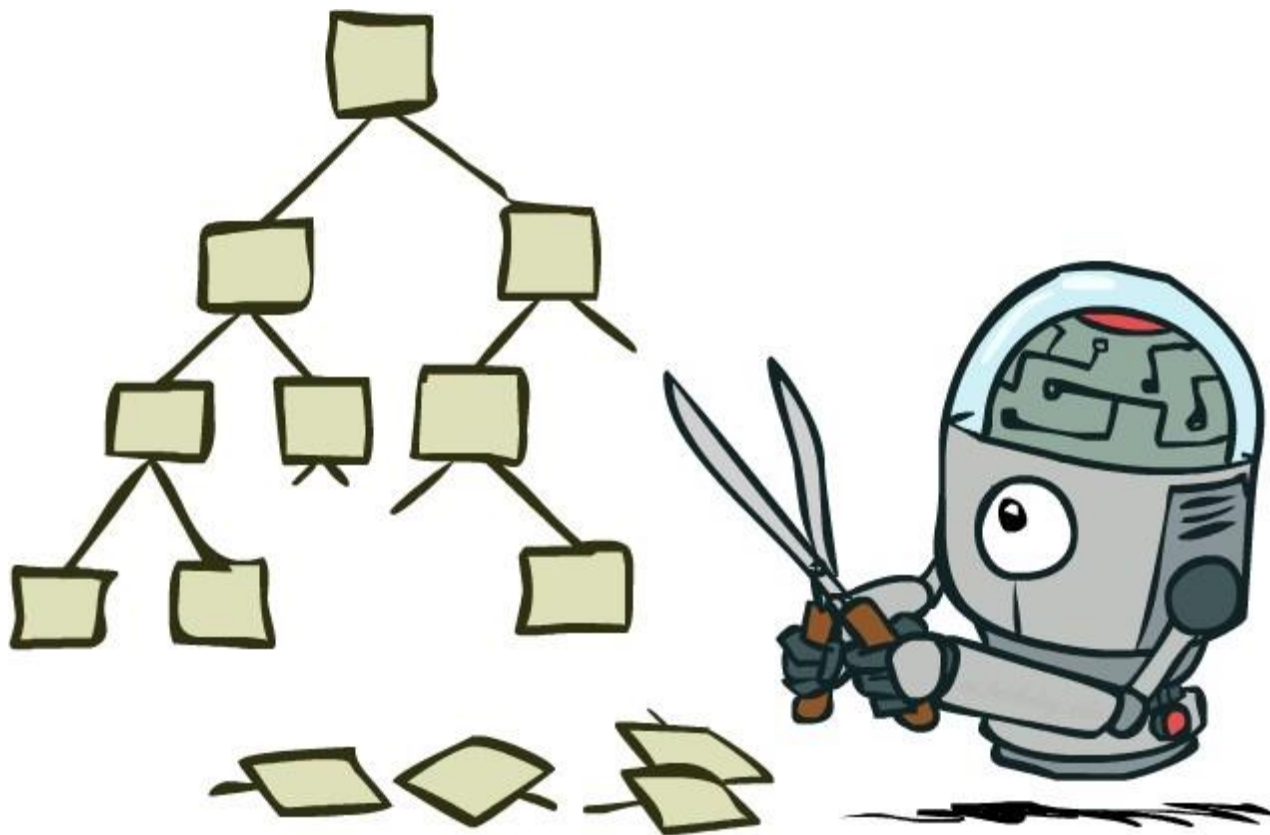
- MINIMAX需要定义效用函数，终止态的效用函数是按输赢或平局确定。非终止态的minimax函数值可以从该路径终止态的输赢结果递归确定。
- 极小极大搜索的交替走步基本思想：轮到电脑（max）走，选极大的minimax值，轮到对手走，选极小的minimax值。
- 极小极大搜索算法需要双方都了解全局的当前局势和对手下一步的所有可能步。

博弈搜索——ALPHABETA算法



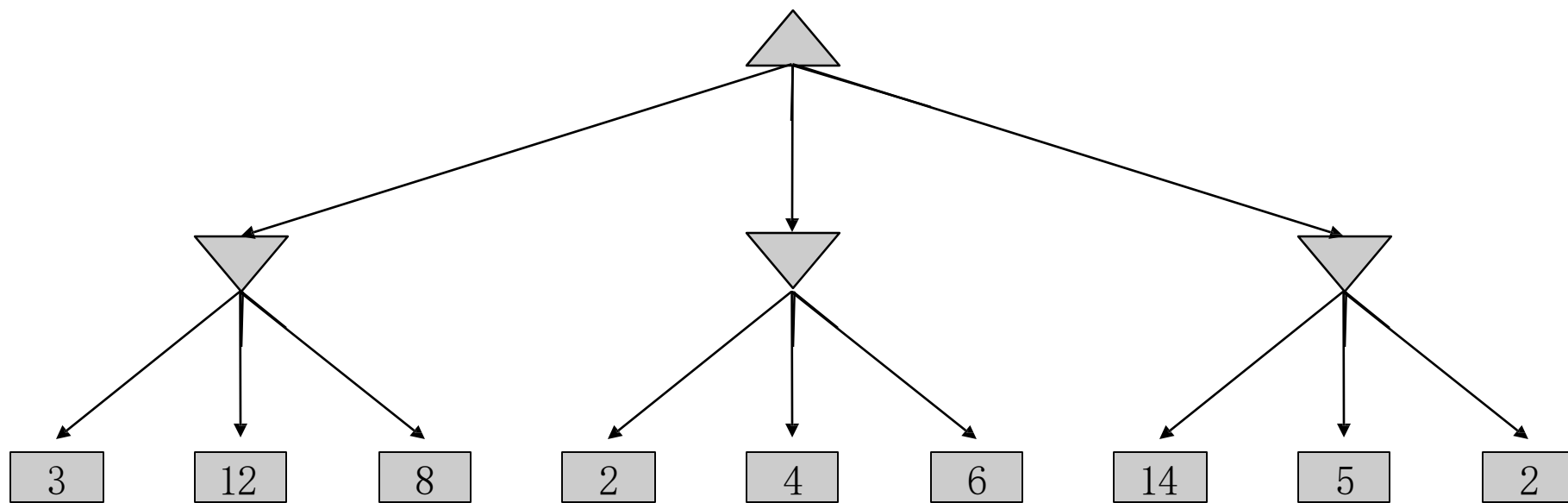
资源有限

博弈搜索——ALPHABETA算法



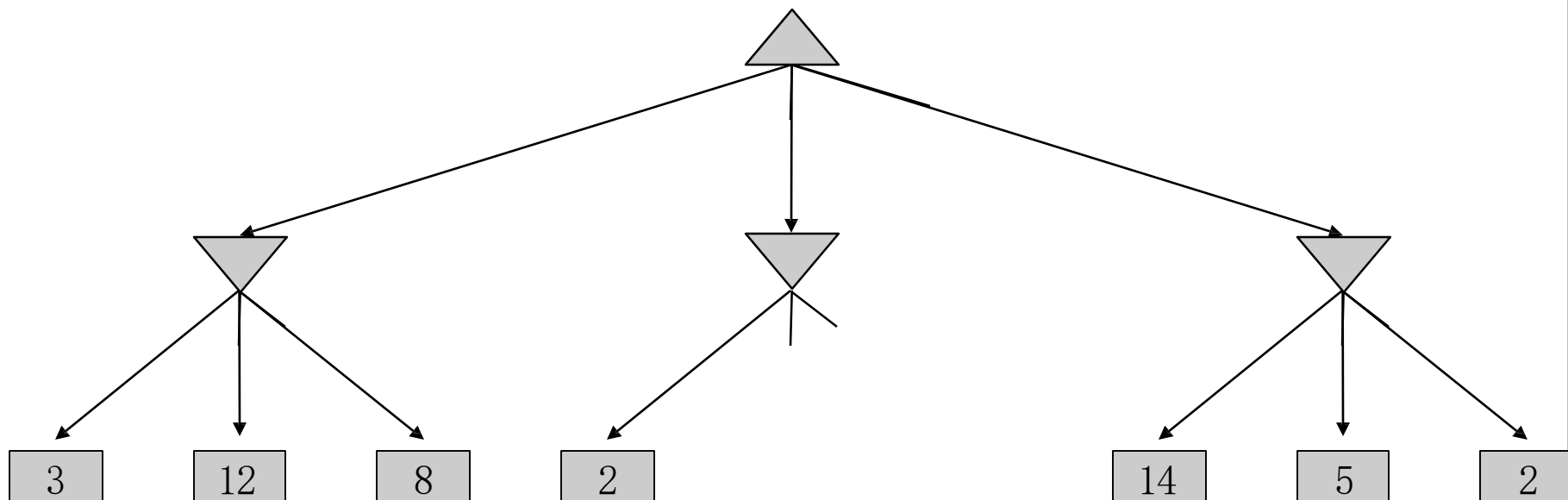
博弈树剪枝

博弈搜索——ALPHABETA算法



博弈树剪枝

博弈搜索——ALPHABETA算法



博弈搜索——ALPHABETA算法

符号	谁在用	含义
α (alpha)	MAX (我方)	Max玩家 目前至少能拿到的最好分数
β (beta)	MIN (对手)	Min玩家 目前最多会让 Max玩家 拿到的分数

剪枝条件 $\alpha \geq \beta$

Max玩家目前至少能拿到的最好分数 $\geq$ 最多会让**Max玩家**拿到的分数



继续算下去已经没必要，因为结果不会更好了

博弈搜索——ALPHABETA算法

新产品发布与竞争对手反制

- **MAX (己方公司):** 目标是选择最佳的定价策略和广告策略，以最大化利润。
- **MIN (竞争对手):** 目标是选择最佳的反制策略（例如：打折或推出竞品），以最小化 MAX 的利润。
- **MAX 决策层 (Level 0):** 选择定价策略 (P_1, P_2)。
- **MIN 决策层 (Level 1):** 竞争对手的反制行动 (C_1, C_2)。
- **MAX 决策层 (Level 2):** 选择广告策略 (A_1, A_2)。
- **终端节点 (Level 3):** 最终的市场利润 (MAX 的得分)。

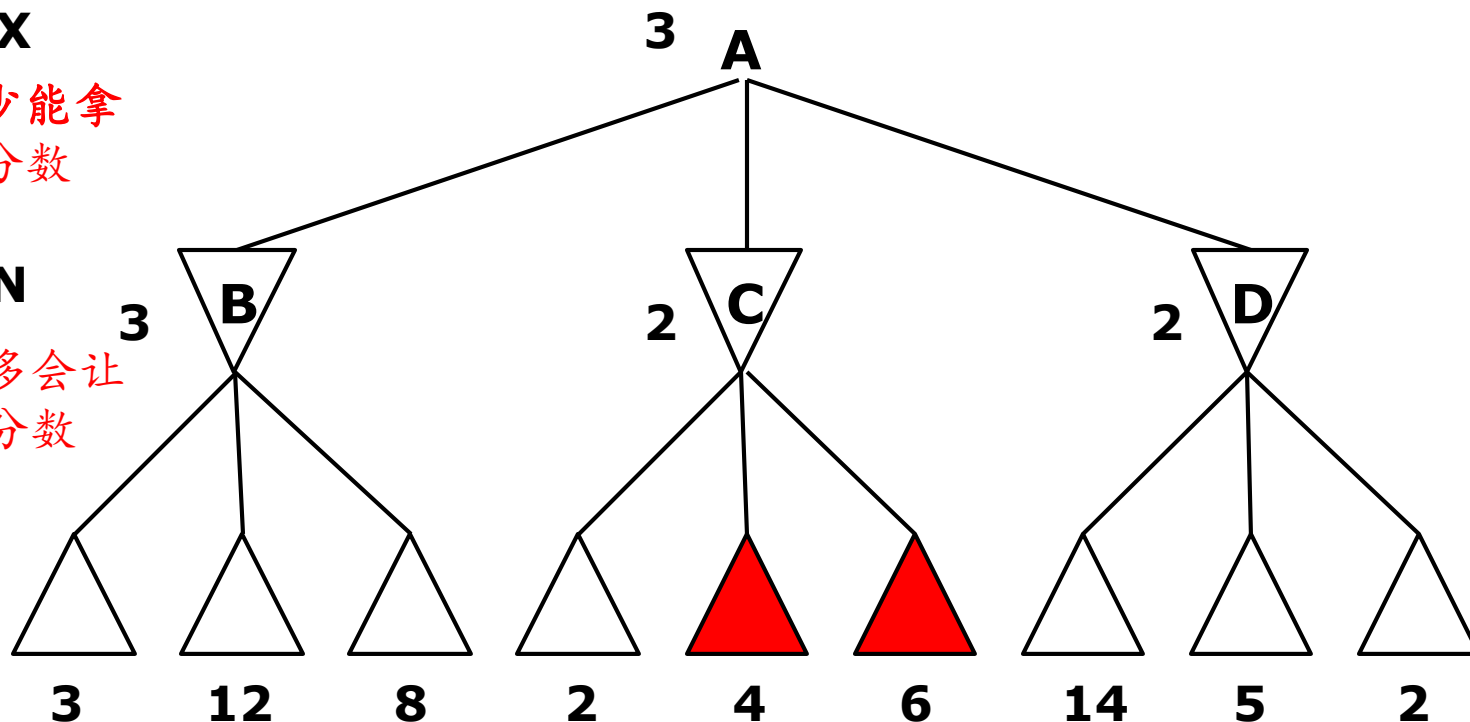
博弈搜索——ALPHABETA算法

MAX

我目前至少能拿到的最好分数

MIN

我目前最多会让你拿到的分数



$$\begin{aligned} \text{MINIMAX}(\text{root}) &= \max(\min(3, 12, 8), \min(2, \textcolor{red}{x}, \textcolor{red}{y}), \min(14, 5, 2)) \\ &= \max(3, \min(2, \textcolor{red}{x}, \textcolor{red}{y}), 2) \\ &= \max(3, z, 2) \quad \text{where } z = \min(2, x, y) \leq 2 \\ &= 3 \end{aligned}$$

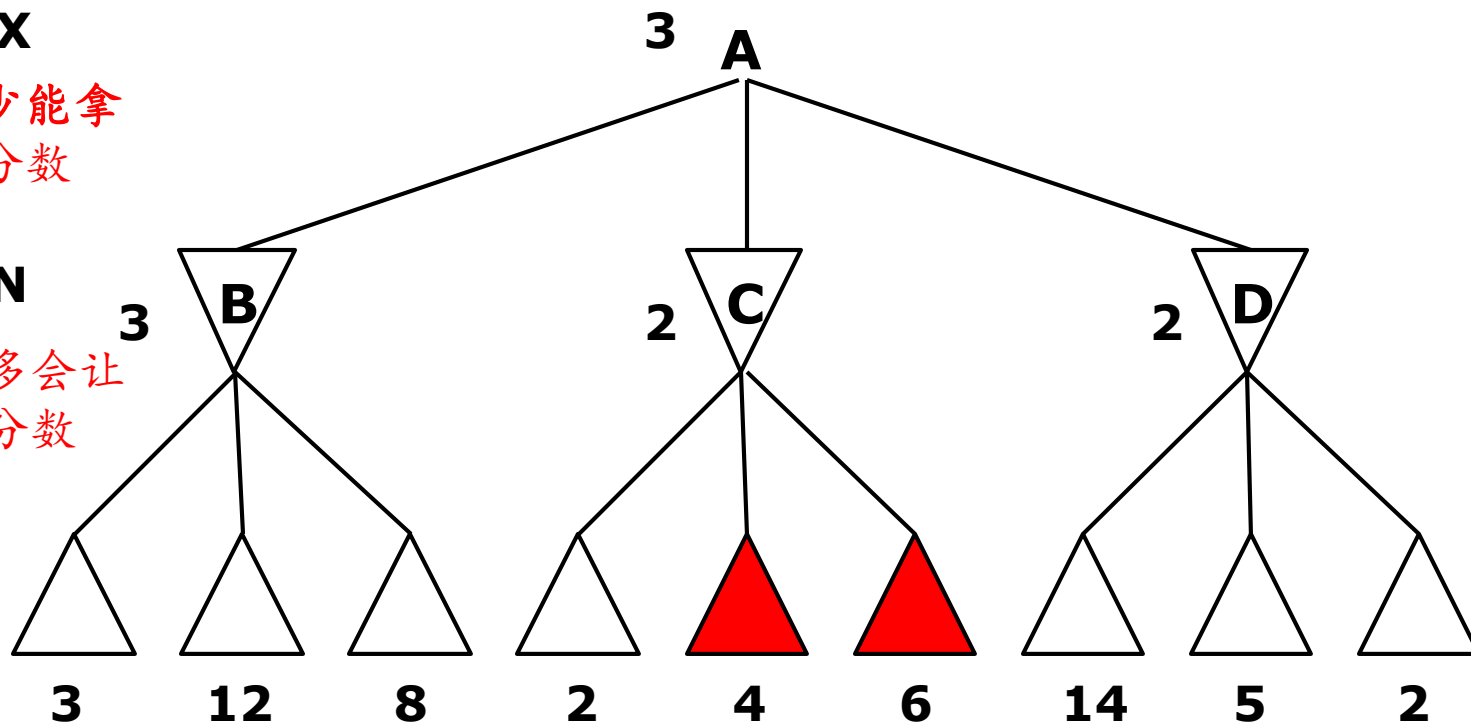
博弈搜索——ALPHABETA算法

MAX

我目前至少能拿到的最好分数

MIN

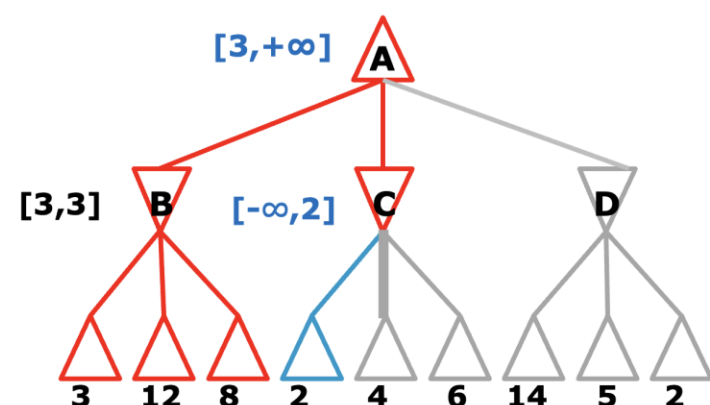
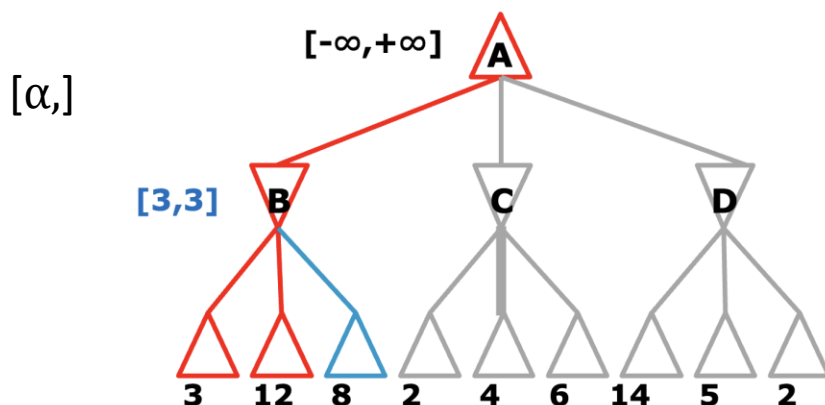
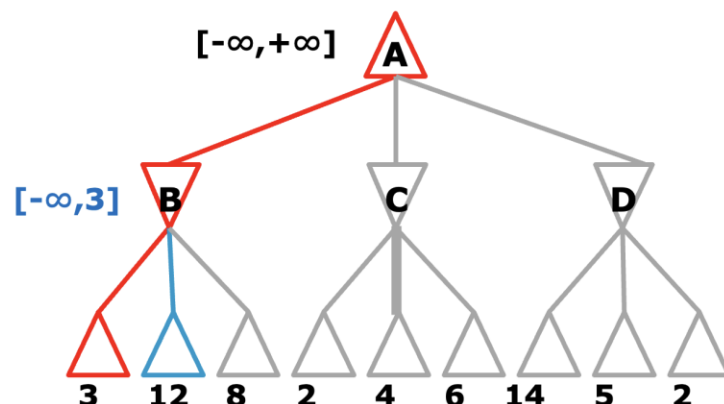
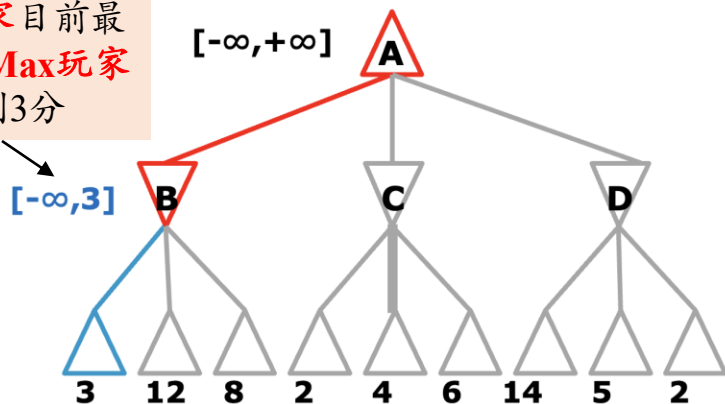
我目前最多会让你拿到的分数



1. 节点B提供了最大下限 $3 (\alpha = 3)$ 。
2. 在评估节点C时，我们发现C节点的最终结果 z 无论如何都不可能大于2。
 - 因为 z 必须是 $\min(2, x, y)$ ，所以 $z \leq 2$ 。
 - 由于 $\alpha = 3$ 已经大于 z 的最大可能值2，所以MAX知道C节点绝对不会是它的最佳选择。
 - 这正是 Alpha-Beta 剪枝的发生之处：我们不必计算 x 和 y 的确切值，因为它们对最终结果没有影响。
3. 节点D提供的结果是2。
4. 最终，根节点A (MAX) 在 $\{3, z(\leq 2), 2\}$ 中选择最大值，结果为3。

博弈搜索——ALPHABETA算法

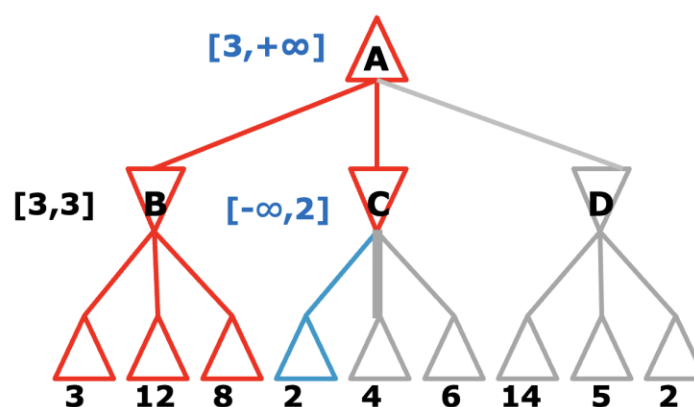
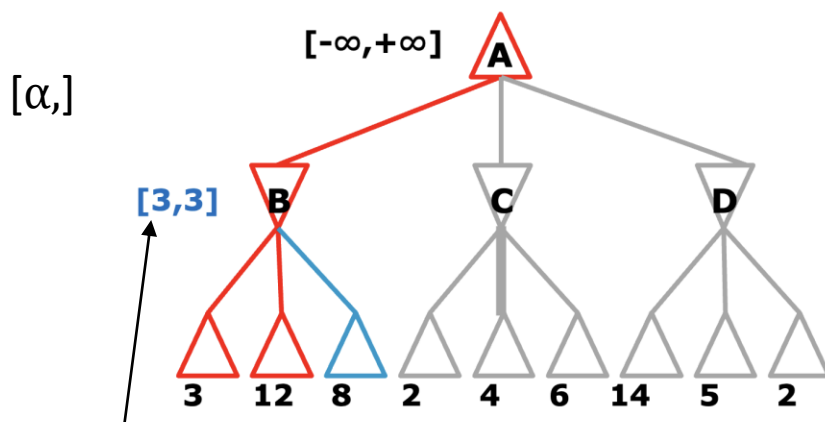
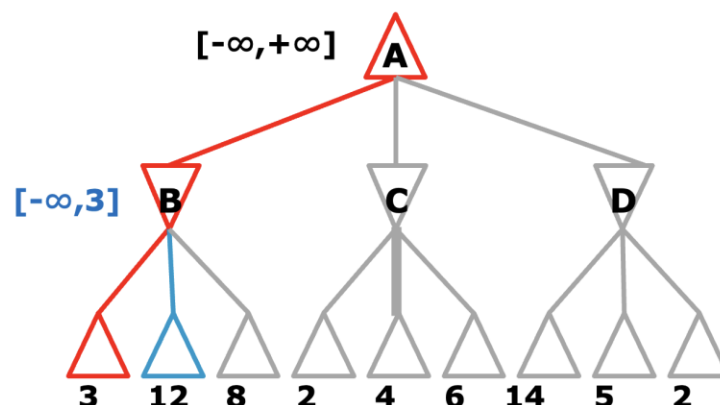
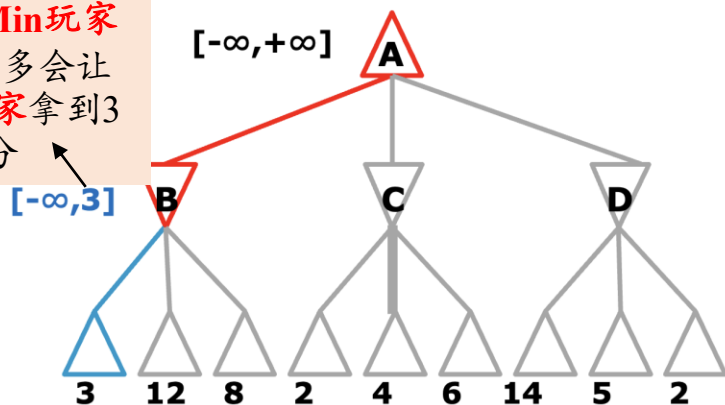
Min玩家目前最多会让Max玩家拿到3分



每个结点都标有其值的范围

博弈搜索——ALPHABETA算法

Beta: Min玩家
目前最多会让
Max玩家拿到3
分

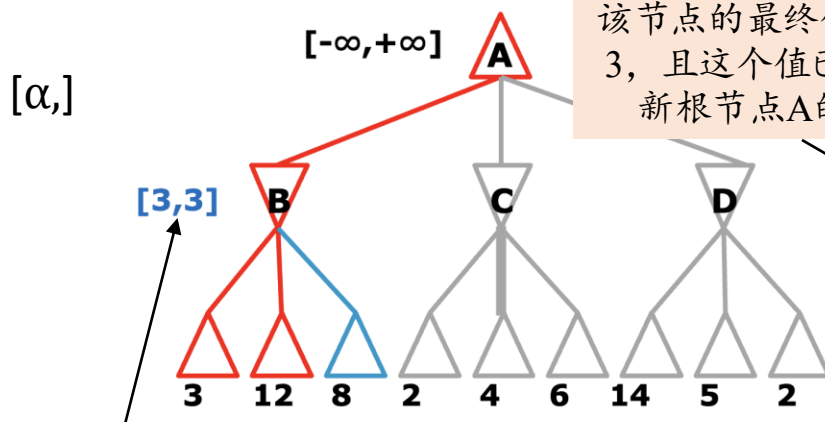
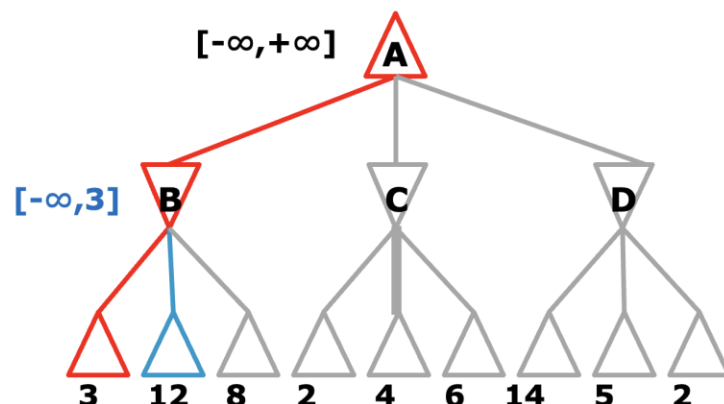
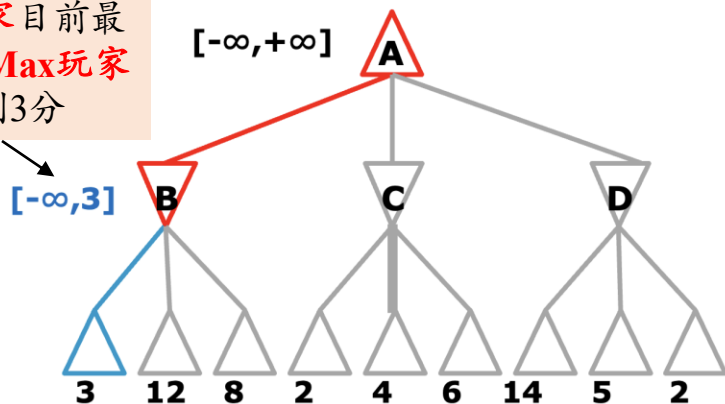


Alpha: Max玩家
目前至少能拿到的最
好分数

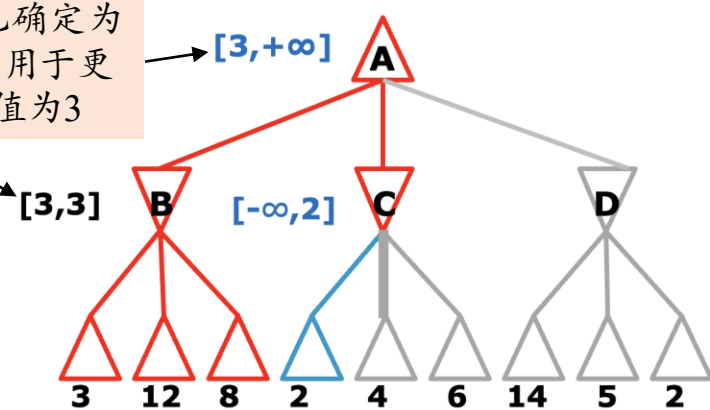
每个结点都标有其值的范围

博弈搜索——ALPHABETA算法

Min玩家目前最多会让Max玩家拿到3分



该节点的最终值已确定为3，且这个值已经用于更新根节点A的 α 值为3

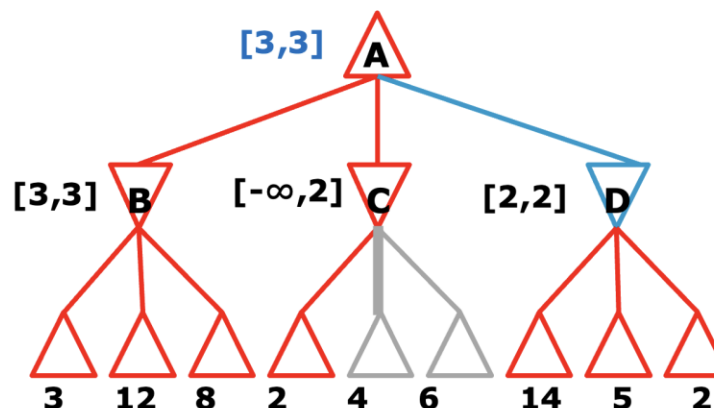
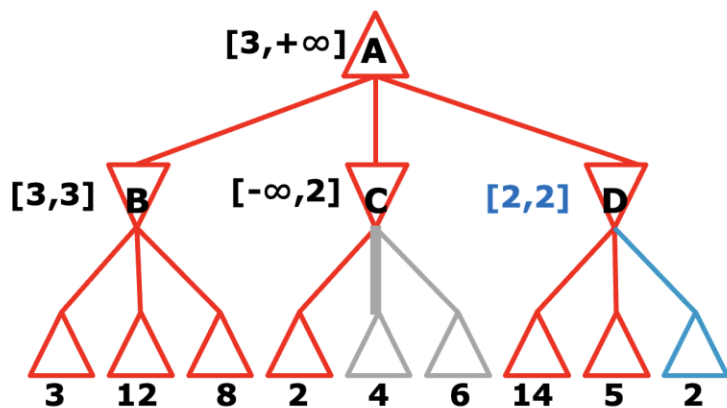
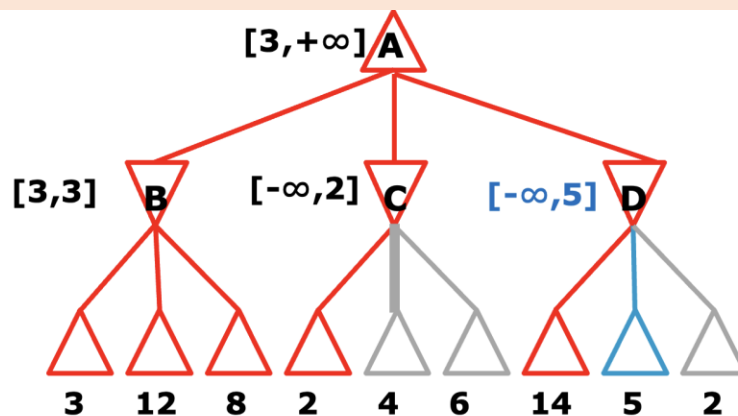
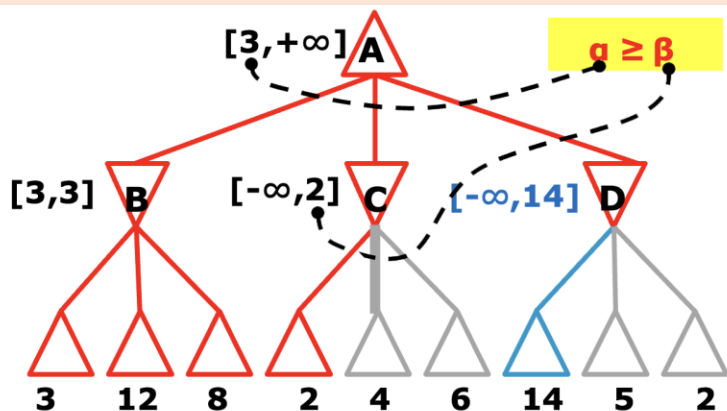


Max玩家目前至少能拿到的最好分数

每个结点都标有其值的范围

博弈搜索——ALPHABETA算法

Max玩家 目前至少能拿到的最好分数 $\geq$ 最多会让 **Max玩家** 拿到的分数



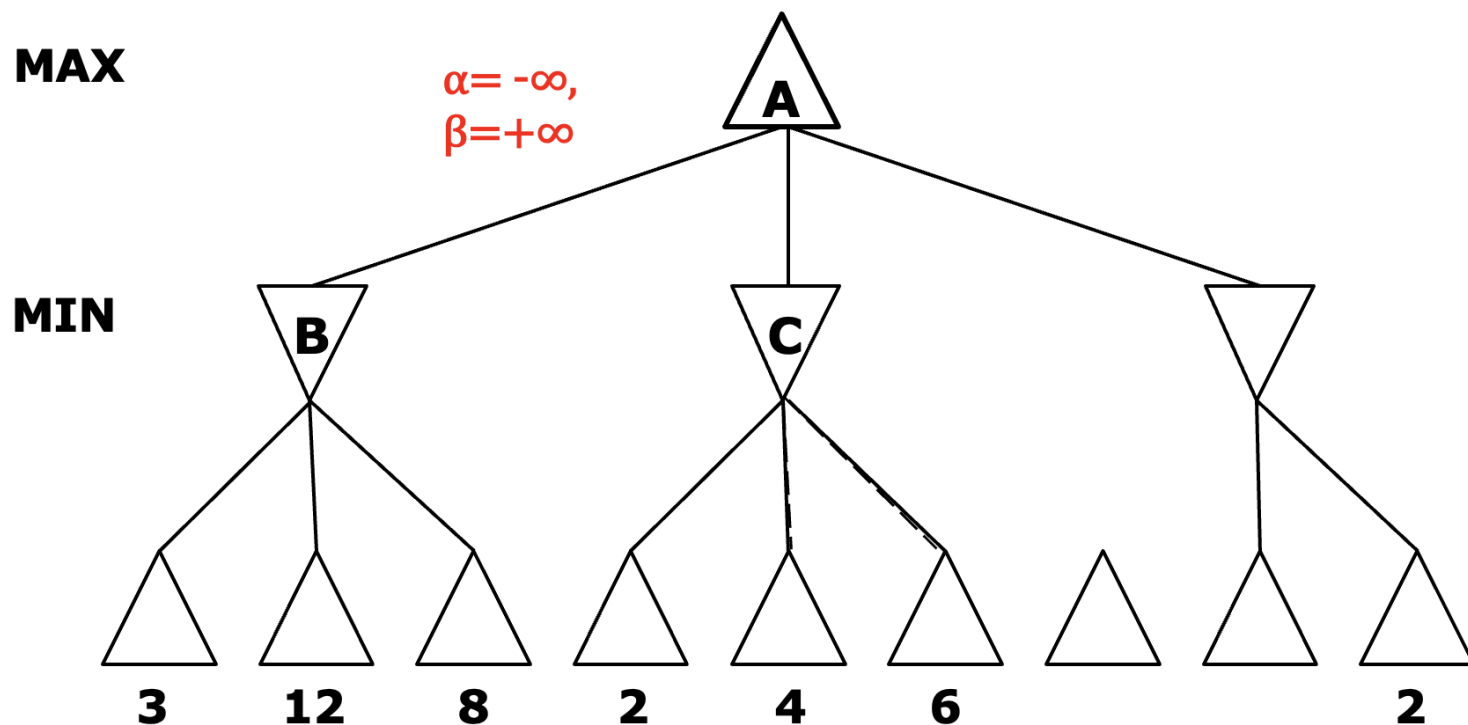
博弈搜索——ALPHABETA算法

```
function Alpha-Beta-Search(state) returns an action  
   $v \leftarrow \text{Max-Value}(\text{state}, -\infty, +\infty)$   
  return the action in Actions(state) with value  $v$ 
```

```
function Max-Value(state,  $\alpha, \beta$ ) returns a utility value  
  if Terminal-Test(state) then return Utility(state)  
  for each  $a$  in Actions(state) do  
     $\alpha = \text{Max}(\alpha, \text{Min-Value}(\text{Result}(s, a), \alpha, \beta))$   
    if  $\alpha \geq \beta$  then return  $\alpha$   
  return  $\alpha$ 
```

```
function Min-Value(state,  $\alpha, \beta$ ) returns a utility value  
  if Terminal-Test(state) then return Utility(state)  
  for each  $a$  in Actions(state) do  
     $\beta = \text{Min}(\beta, \text{Max-Value}(\text{Result}(s, a), \alpha, \beta))$   
    if  $\alpha \geq \beta$  then return  $\beta$   
  return  $\beta$ 
```

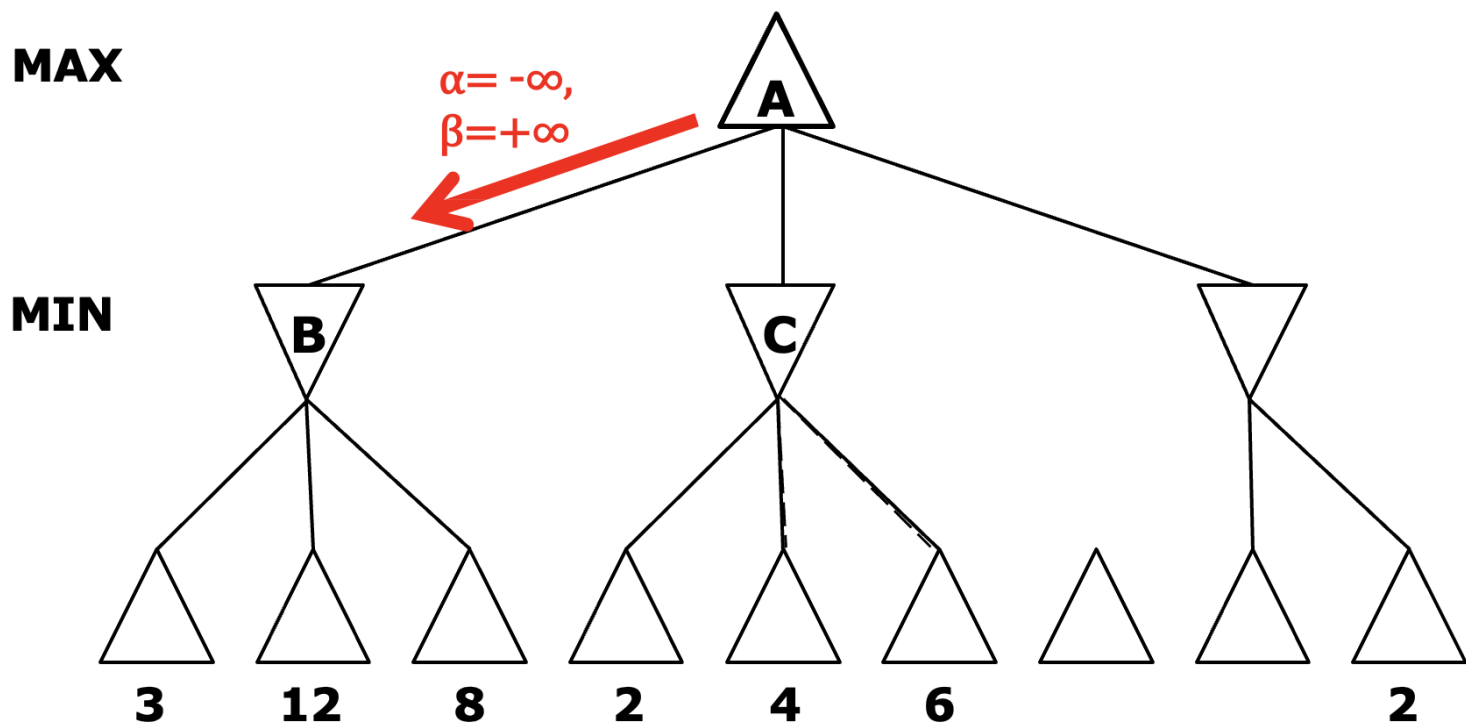
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的**MAX**的最佳值（即极大值）

β = 到目前为止路径上发现的**MIN**的最佳值（即极小值）

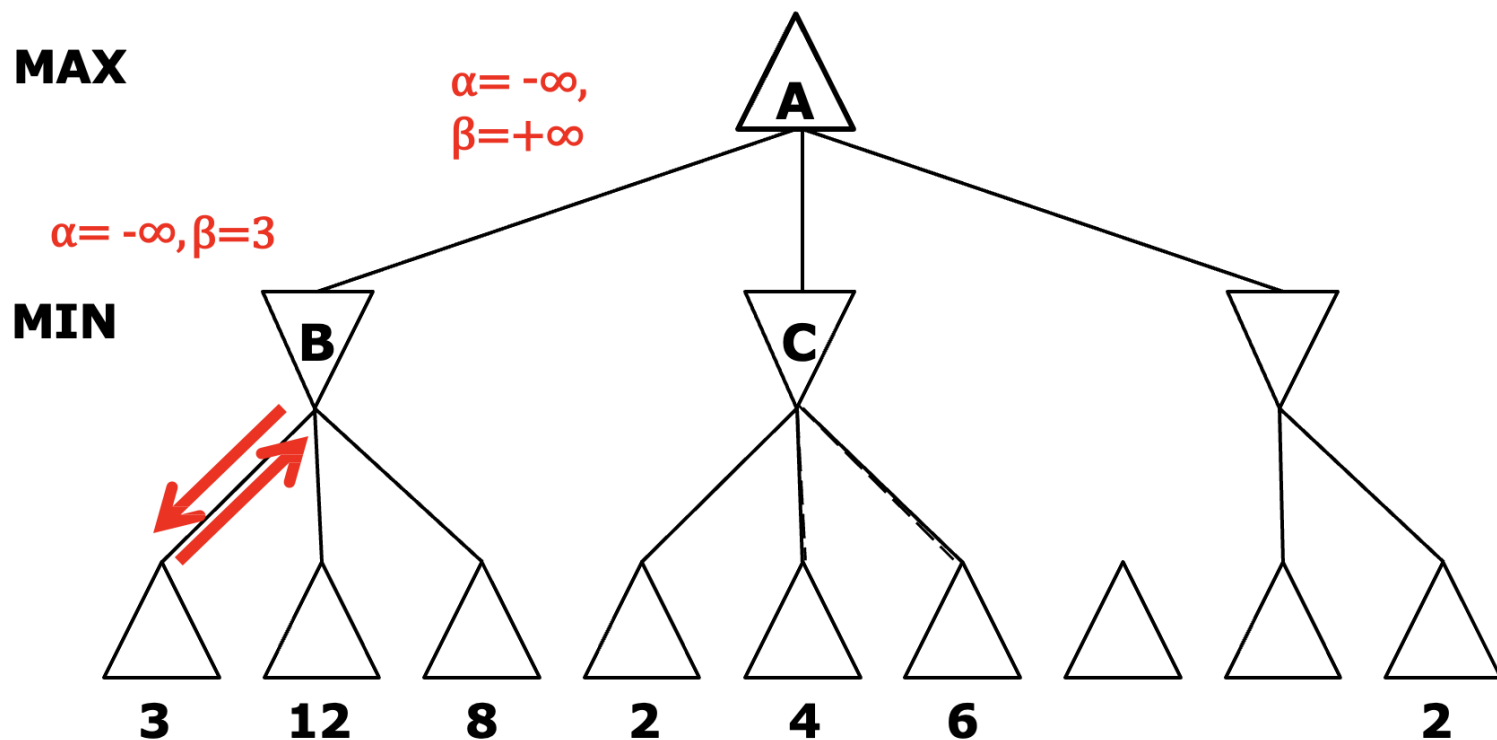
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的**MAX**的最佳值（即极大值）

β = 到目前为止路径上发现的**MIN**的最佳值（即极小值）

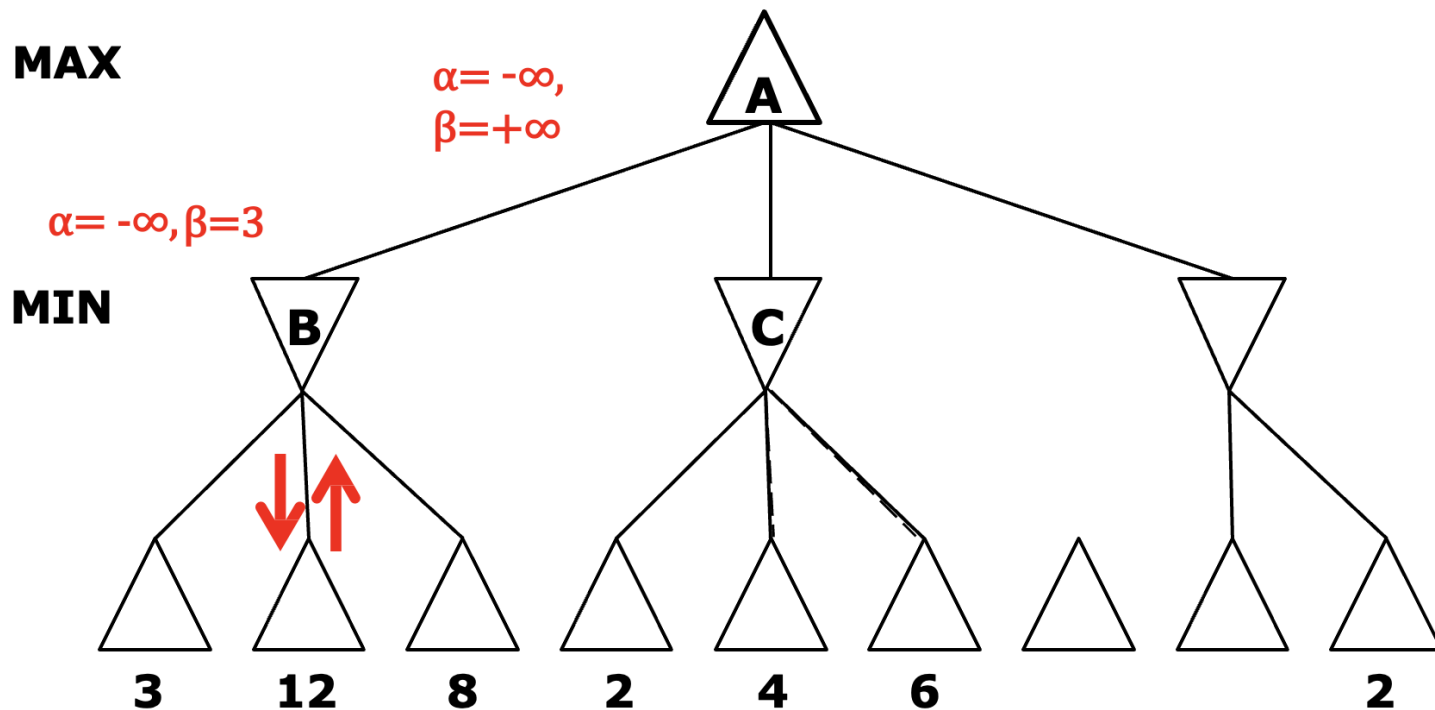
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的**MAX**的最佳值（即极大值）

β = 到目前为止路径上发现的**MIN**的最佳值（即极小值）

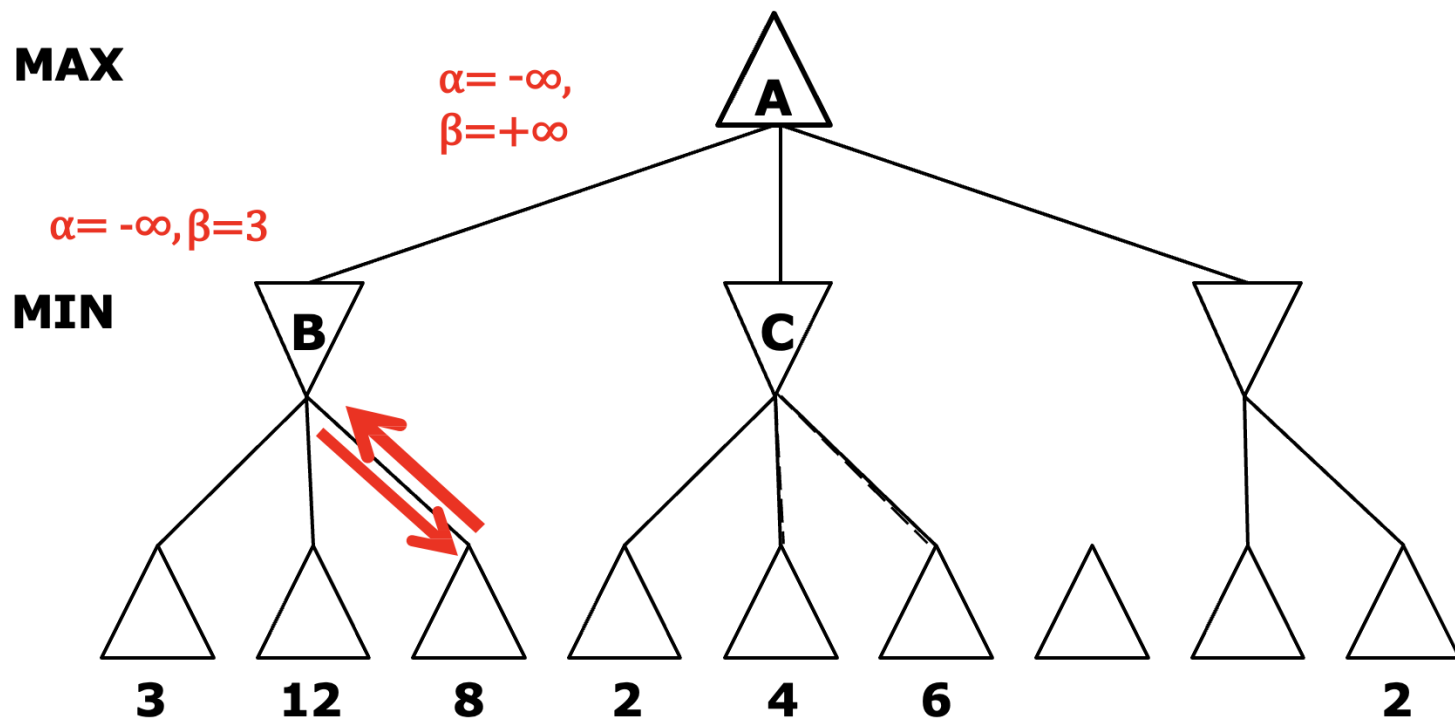
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的**MAX**的最佳值（即极大值）

β = 到目前为止路径上发现的**MIN**的最佳值（即极小值）

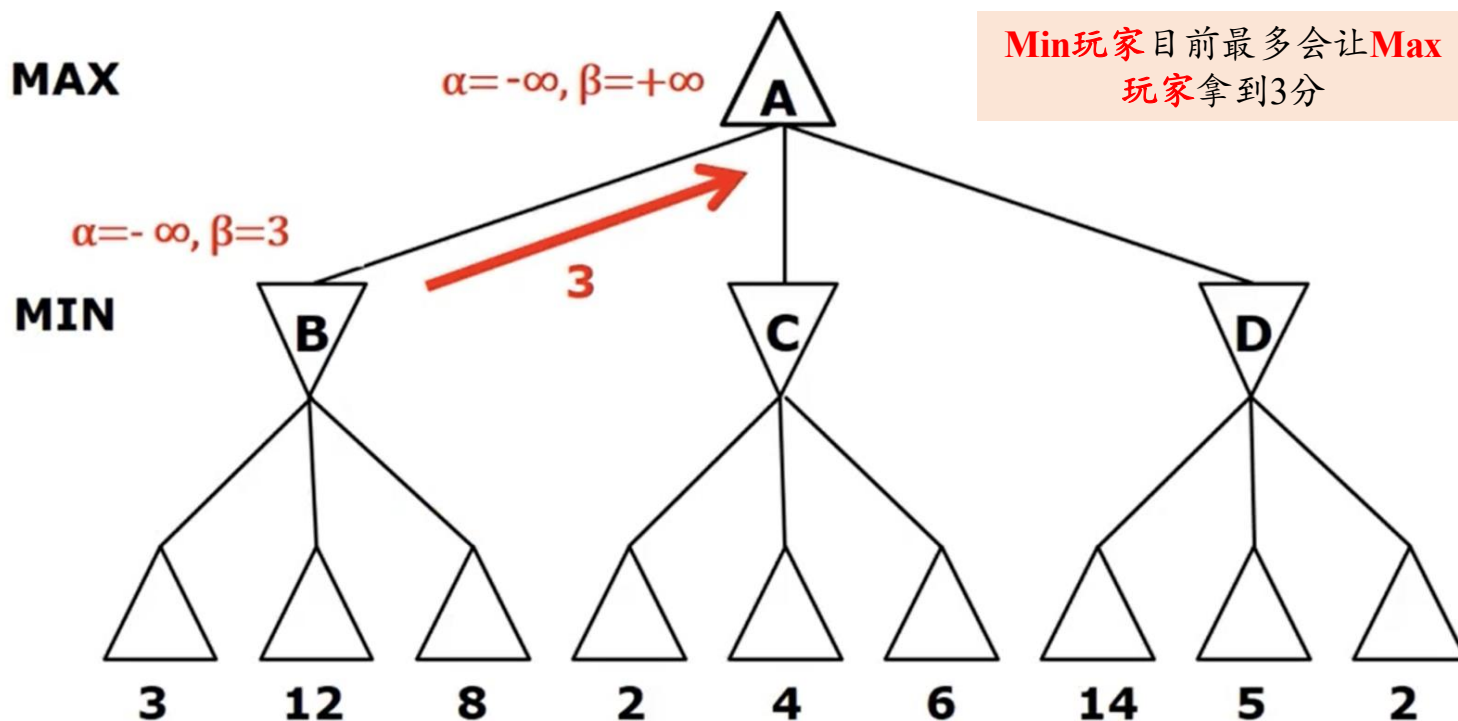
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的**MAX**的最佳值（即极大值）

β = 到目前为止路径上发现的**MIN**的最佳值（即极小值）

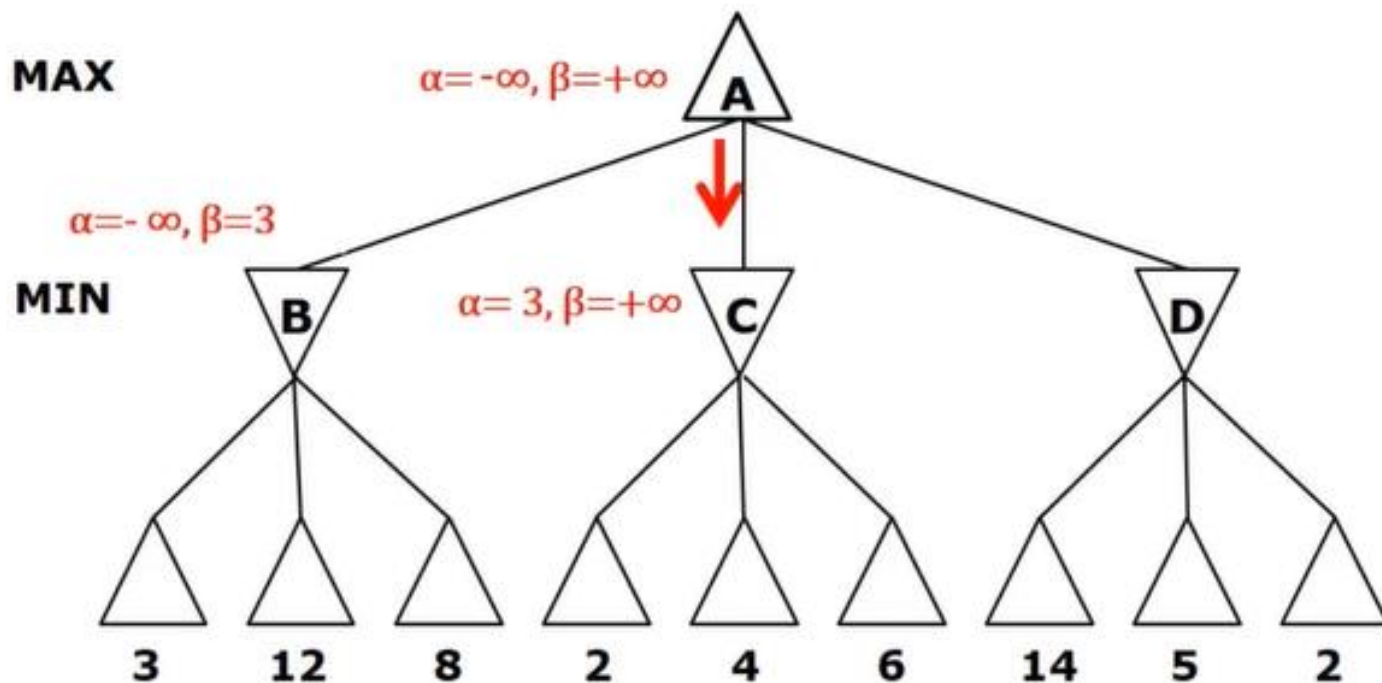
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的**MAX**的最佳值（即极大值）

β = 到目前为止路径上发现的**MIN**的最佳值（即极小值）

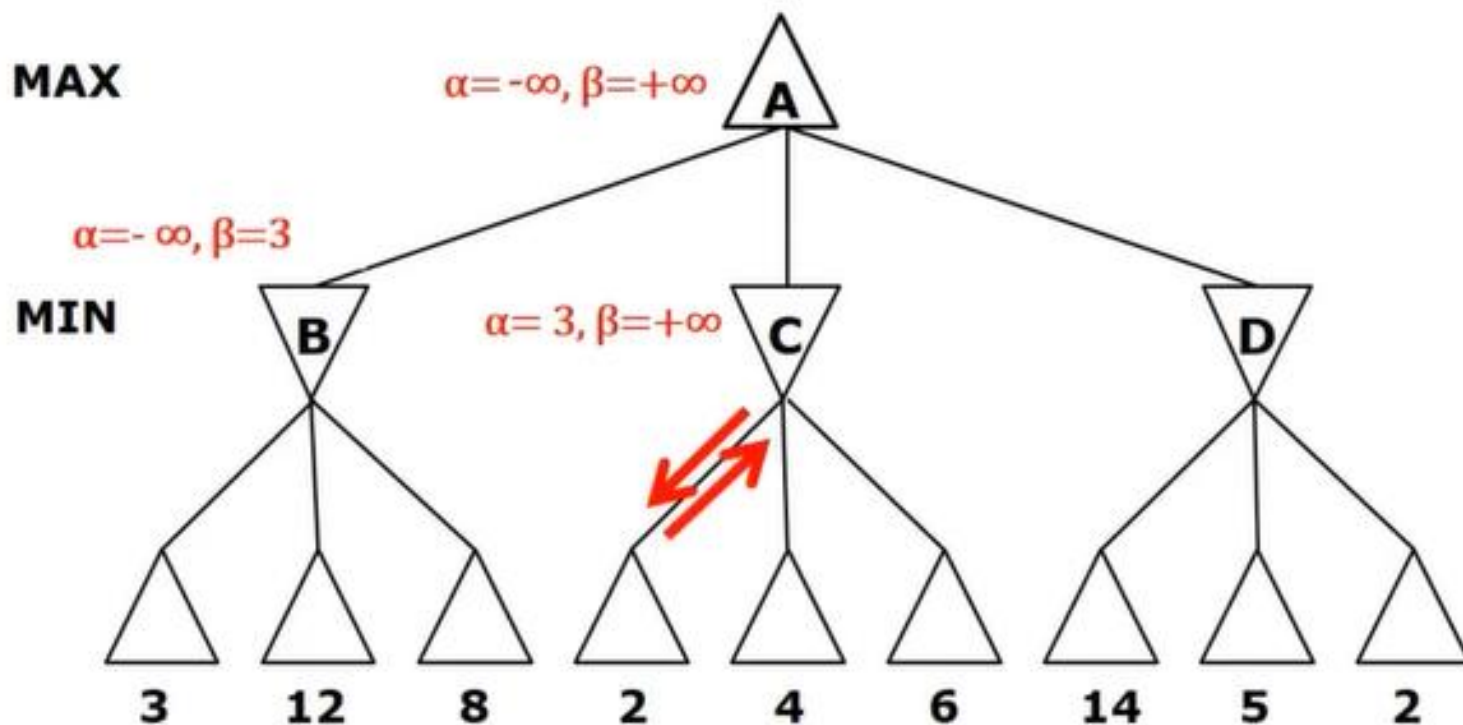
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的**MAX**的最佳值（即极大值）

β = 到目前为止路径上发现的**MIN**的最佳值（即极小值）

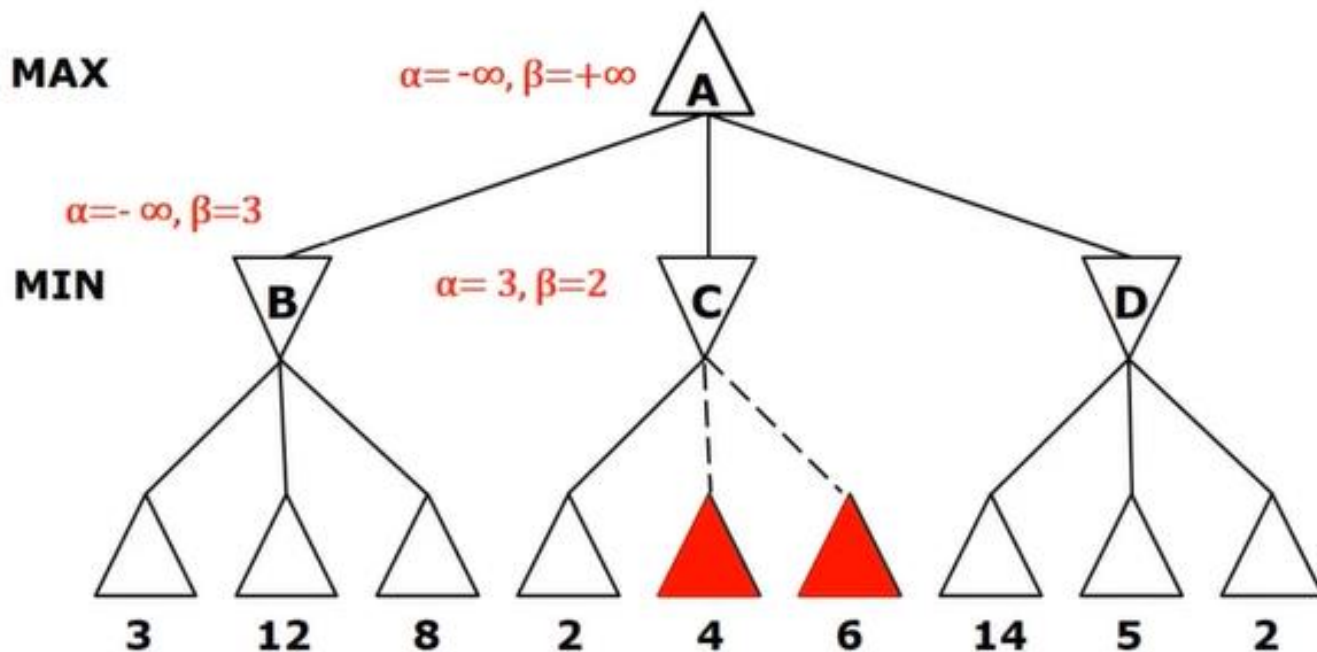
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的MAX的最佳值（即极大值）

β = 到目前为止路径上发现的MIN的最佳值（即极小值）

博弈搜索——ALPHABETA算法

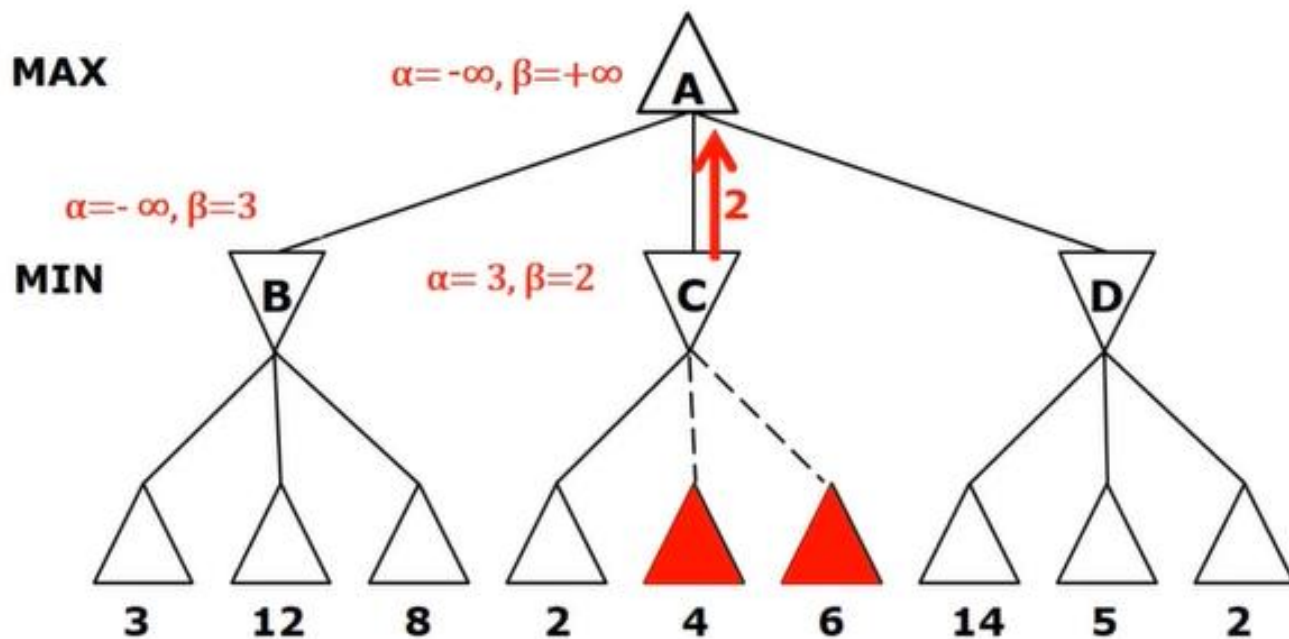


触发 $\alpha \geq \beta$ 剪枝，4和6的计算被跳过

α = 到目前为止路径上发现的MAX的最佳值（即极大值）

β = 到目前为止路径上发现的MIN的最佳值（即极小值）

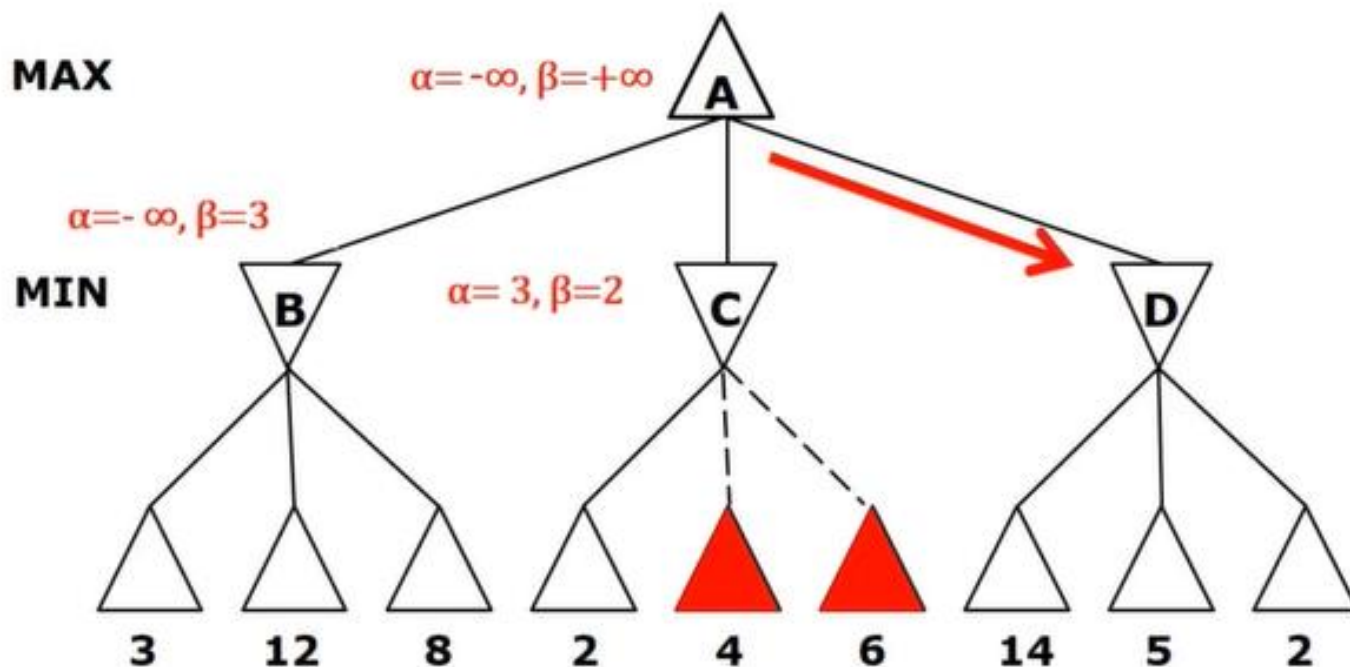
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的MAX的最佳值（即极大值）

β = 到目前为止路径上发现的MIN的最佳值（即极小值）

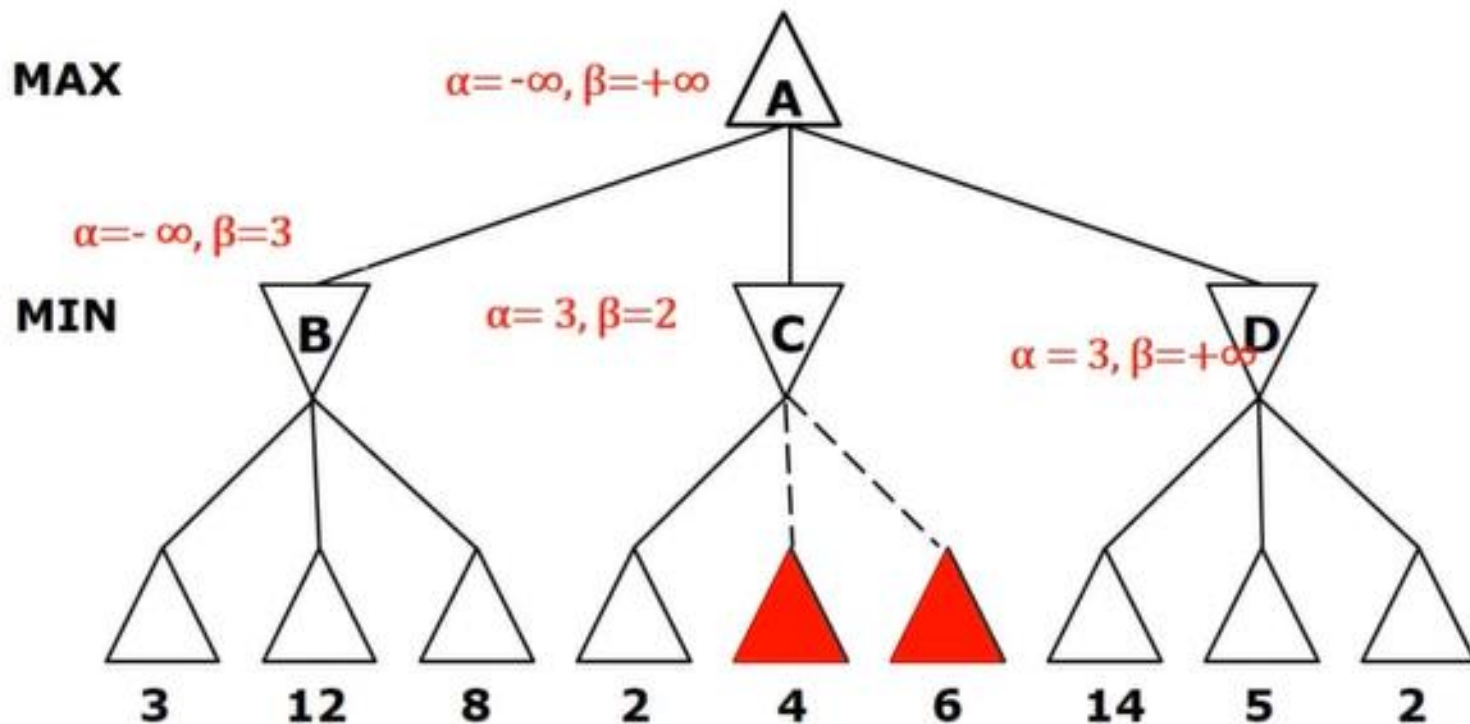
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的**MAX**的最佳值（即极大值）

β = 到目前为止路径上发现的**MIN**的最佳值（即极小值）

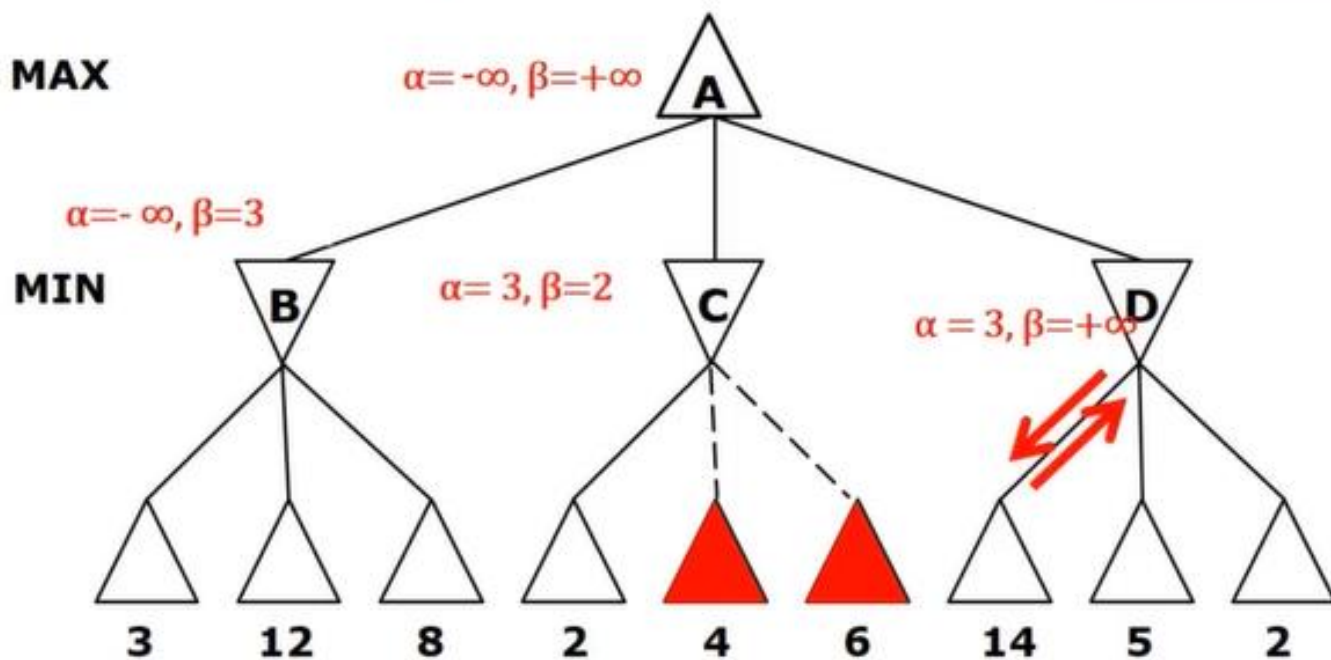
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的MAX的最佳值（即极大值）

β = 到目前为止路径上发现的MIN的最佳值（即极小值）

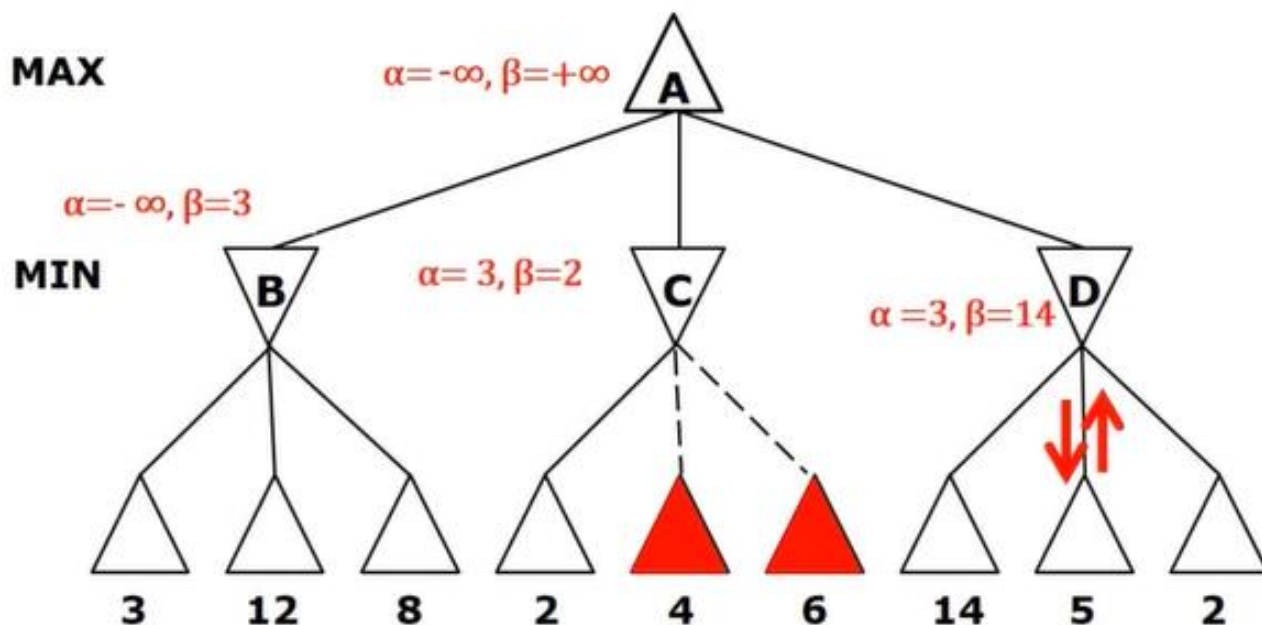
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的**MAX**的最佳值（即极大值）

β = 到目前为止路径上发现的**MIN**的最佳值（即极小值）

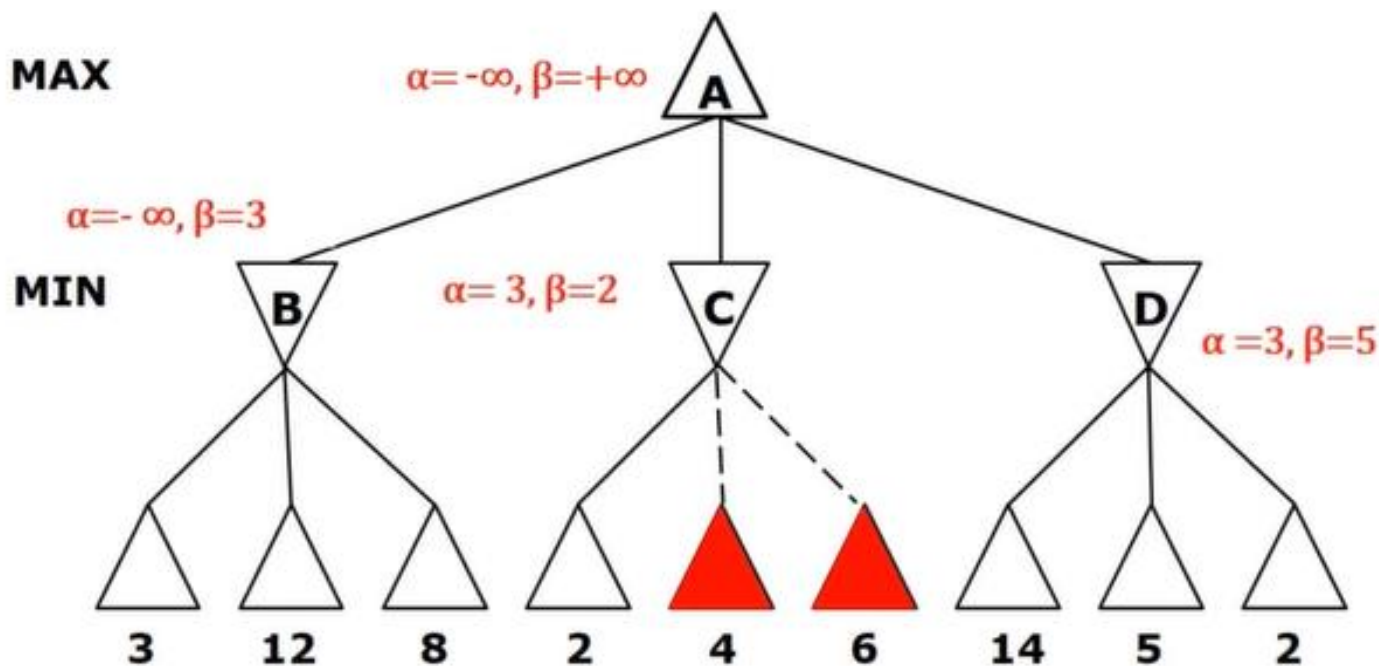
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的MAX的最佳值（即极大值）

β = 到目前为止路径上发现的MIN的最佳值（即极小值）

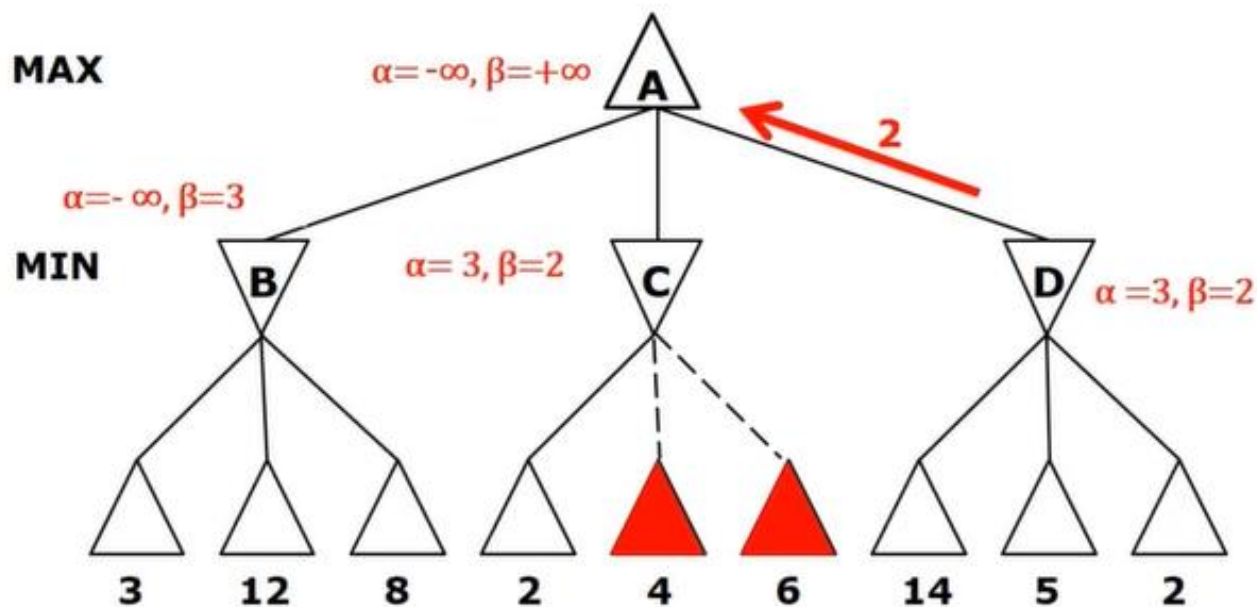
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的MAX的最佳值（即极大值）

β = 到目前为止路径上发现的MIN的最佳值（即极小值）

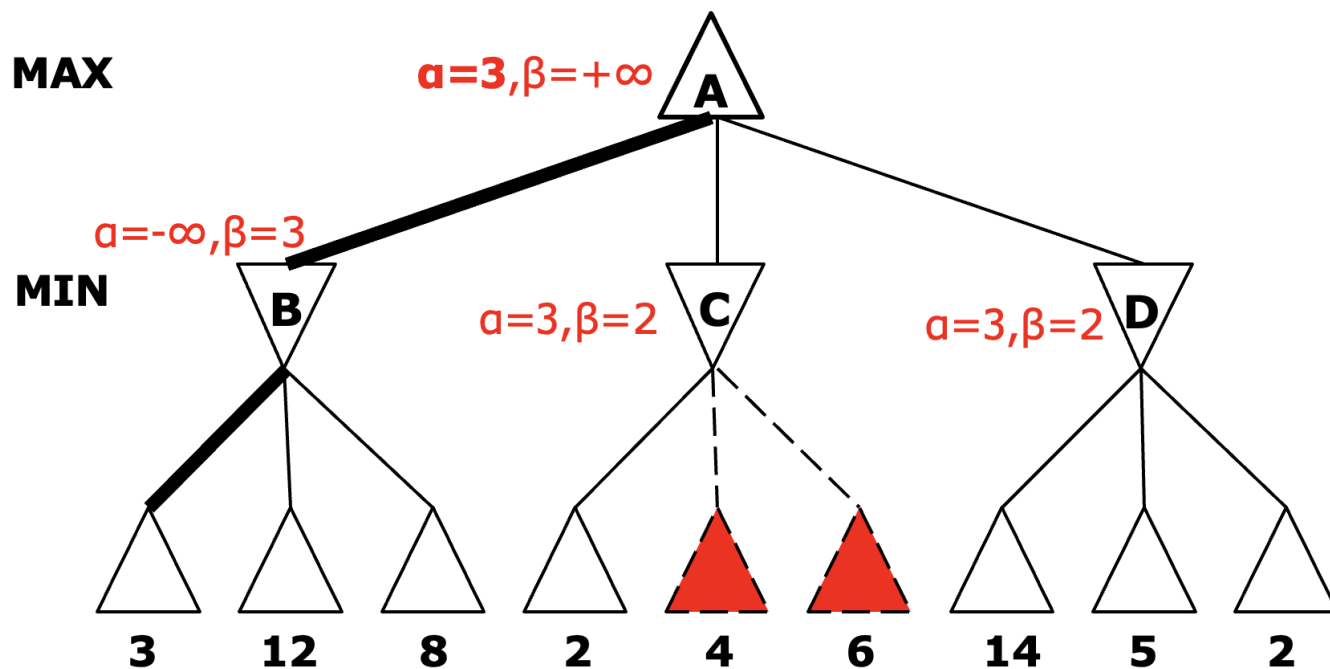
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的MAX的最佳值（即极大值）

β = 到目前为止路径上发现的MIN的最佳值（即极小值）

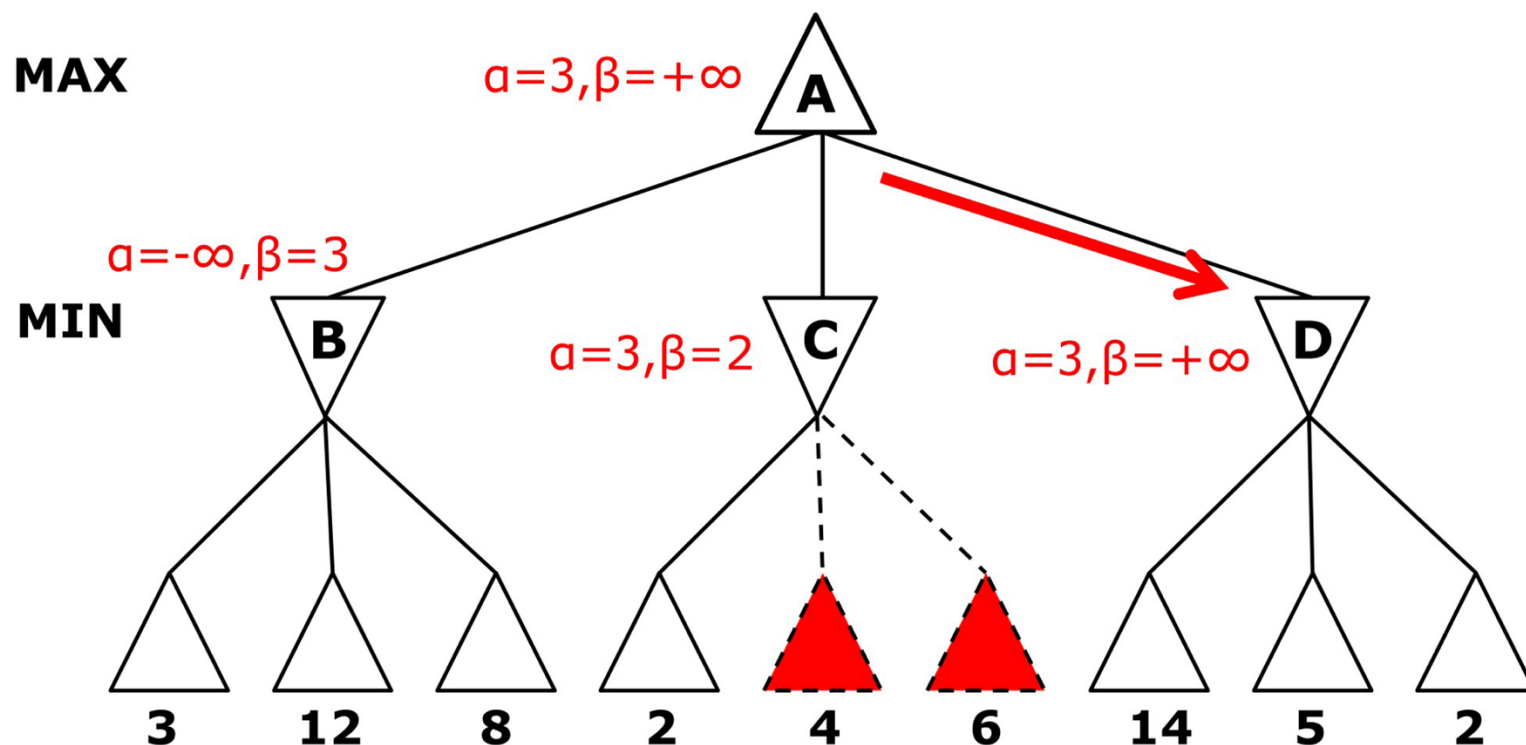
博弈搜索——ALPHABETA算法



α = 到目前为止路径上发现的**MAX**的最佳值（即极大值）

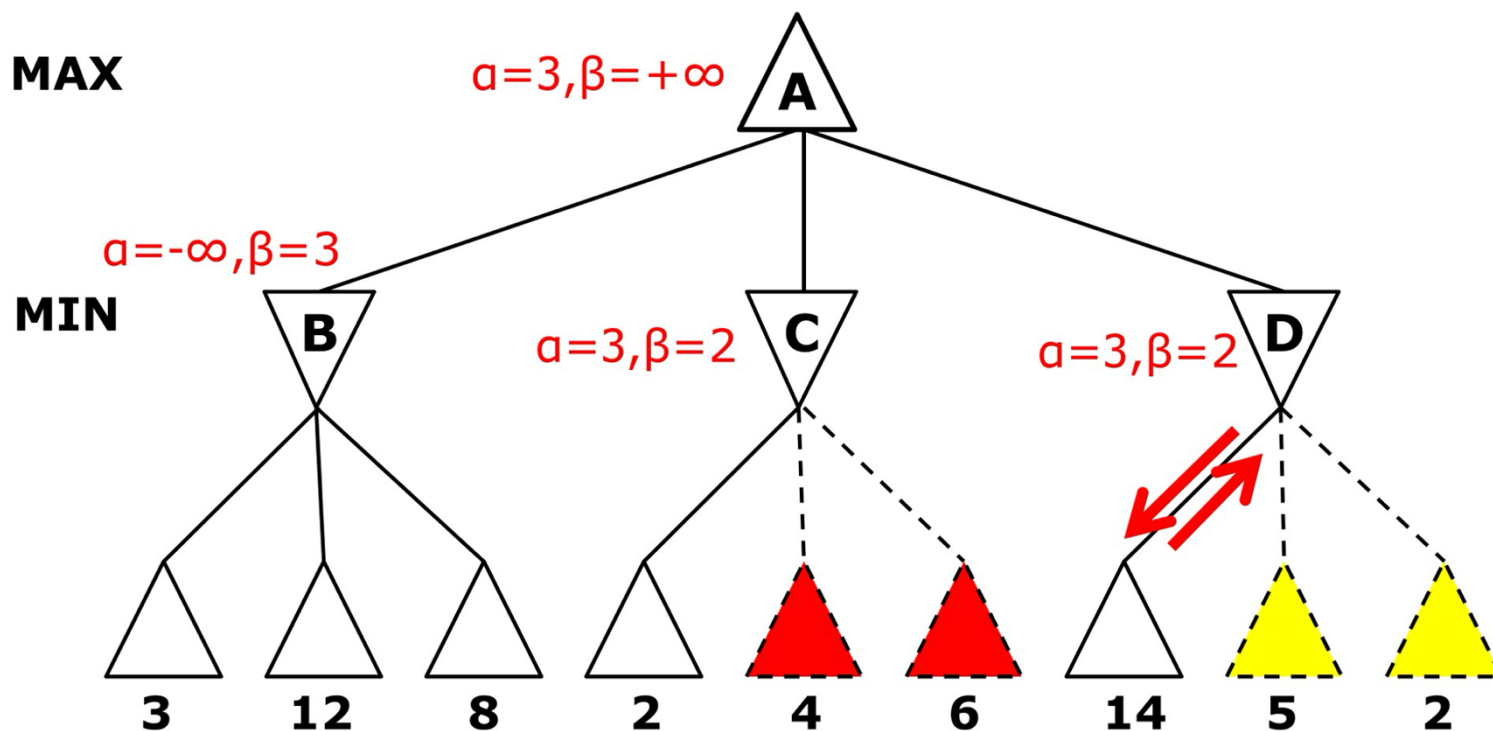
β = 到目前为止路径上发现的**MIN**的最佳值（即极小值）

博弈搜索——ALPHABETA算法



如果改变顺序：2 5 14

博弈搜索——ALPHABETA算法



如果改变顺序：2 5 14

最好将MIN结点的MAX子结点按递增的备份值排序

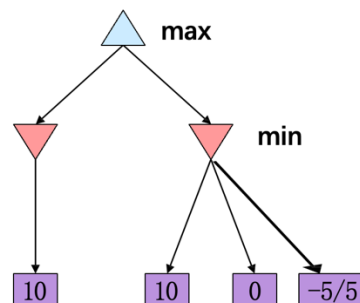
ALPHA-BETA剪枝特性

- 修剪对根算出的minimax值**没有影响**！
- 中间结点的值可能是错误的（因为剪枝提前停止了一些分支的搜索）
 - 重要提示：根的子级可能具有错误的值

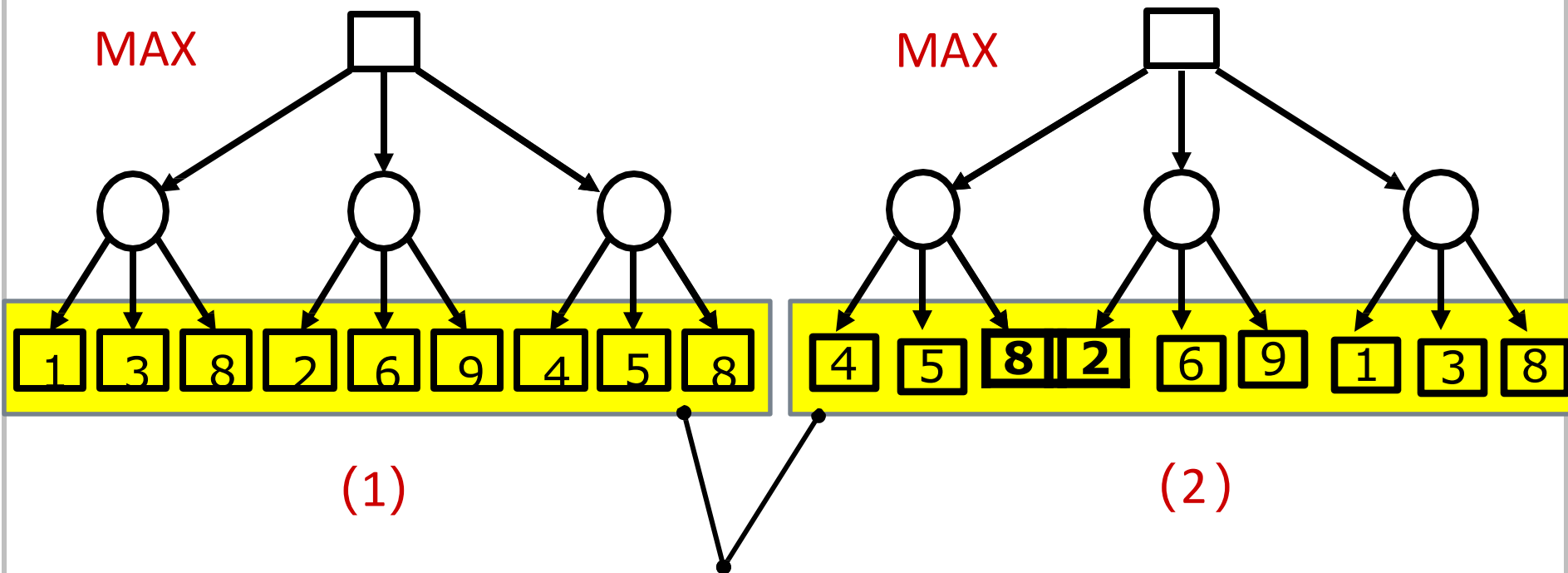
- 好的子结点顺序可以提高修剪效率

- 通过“完美排序”：

- 时间复杂度降低到 $O(b^{m/2})$
- 可解决的深度加倍！
- 完整搜索，例如 象棋，仍然没有解决——搜索空间太大了！

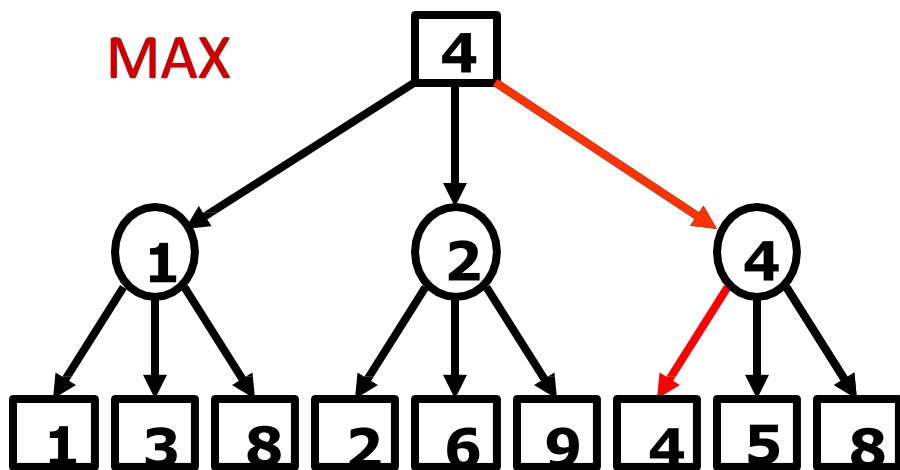


移动顺序

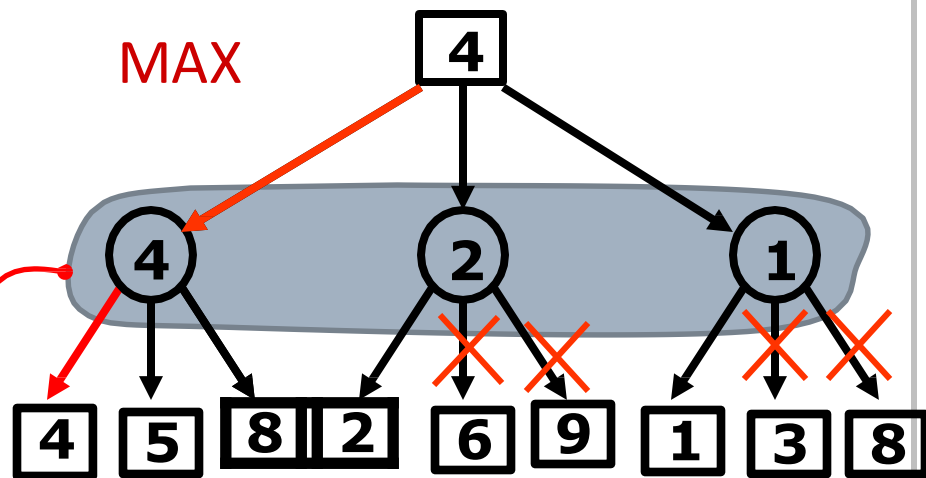


MIN结点的MAX子结点按递增排序

移动顺序



(1)



(2)

最好将MIN结点的MAX子结点按递增的备份值排序

移动顺序

- 假设均匀分支系数为 b 的博弈树
- Minimax 检查 $O(b^m)$ 结点, alpha-beta最坏的情况下也是 $O(b^m)$
- 在以下情况下, alpha-beta的增益最大:
 - MAX结点的MIN子结点按递减的备份值排序
 - MIN结点的MAX子结点按递增的备份值排序
 - 则 alpha-beta检查 $O(b^{m/2})$ 结点
- 如果结点是随机排序的, 则alpha-beta检查的平均结点数约为 $\sim O(b^{3m/4})$

移动顺序

我们常常在有限的时间内无法到达目标结点！

- (1) 当搜索到有限的深度时，只需切断并用EVAL () 替换Utility ()
- (2) 迭代加深算法

移动顺序

- 应该尽早截断搜索
- 用估计棋局效用值的启发式评估函数EVAL取代效用函数，用决策 何时EVAL的截断测试(**cutoff test**)取代终止测试

$$\text{H-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s) & \text{if CUTOFF-TEST}(s, d) \\ \max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if PLAYER}(s) = \text{MIN}. \end{cases}$$

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if PLAYER}(s) = \text{MIN} \end{cases}$$

评估函数

- 显而易见的是博弈程序的性能严重依赖于评估函数的质量。
- 国际象棋入门书给出各个棋子的子力价值估计
- 兵值1分，马和象值3分，车值5分，后值9 分。其他特征诸如
“是否好兵阵”和“王是否安全”可能值半个兵。这些特征值简单地加在一起就得到了对棋局的评估。

$$\text{EVAL}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s) = \sum_{i=1}^n w_i f_i(s)$$

小结

一、剪枝原理（剪枝是可能的）

MIN 层剪枝条件：

MIN 层某个节点发现自己的子节点 minimax 值 $\leq$ 已访问的同层节点最小 minimax 值，这个节点就不需要再检查其他子节点了。

（更准确地理解：当 $\alpha \geq$ 局部 β 时，即可剪枝。）

二、效率与优化

排序对效率的影响：

MAX 层、MIN 层如果已经具有“好的”顺序可以提高效率，时间复杂度可降低到 $O(bm/2)$ 。

MAX 节点的 MIN 子节点按递减的值排序（最好/最高分的结果先算）。

MIN 节点的 MAX 子节点按递增的值排序（最差/最低分的结果先算）。

平均复杂度：

Alpha-Beta 检查的平均结点数约为 $O(b^{3m/4})$ 。

小结

三、应用与截断

提前截断的用途：

提前截断可以避免在规定时间内找不到终止状态结点的情况。

截断时的函数替换：

提前截断采用 **EVAL 函数**（评估函数）代替 **utility 函数**（效用函数）。

（注：效用函数用于终局状态，评估函数用于非终局状态的中间评估。）